

MIT AI2 204

IoT with MIT App Inventor

Fundamental

X. Tang

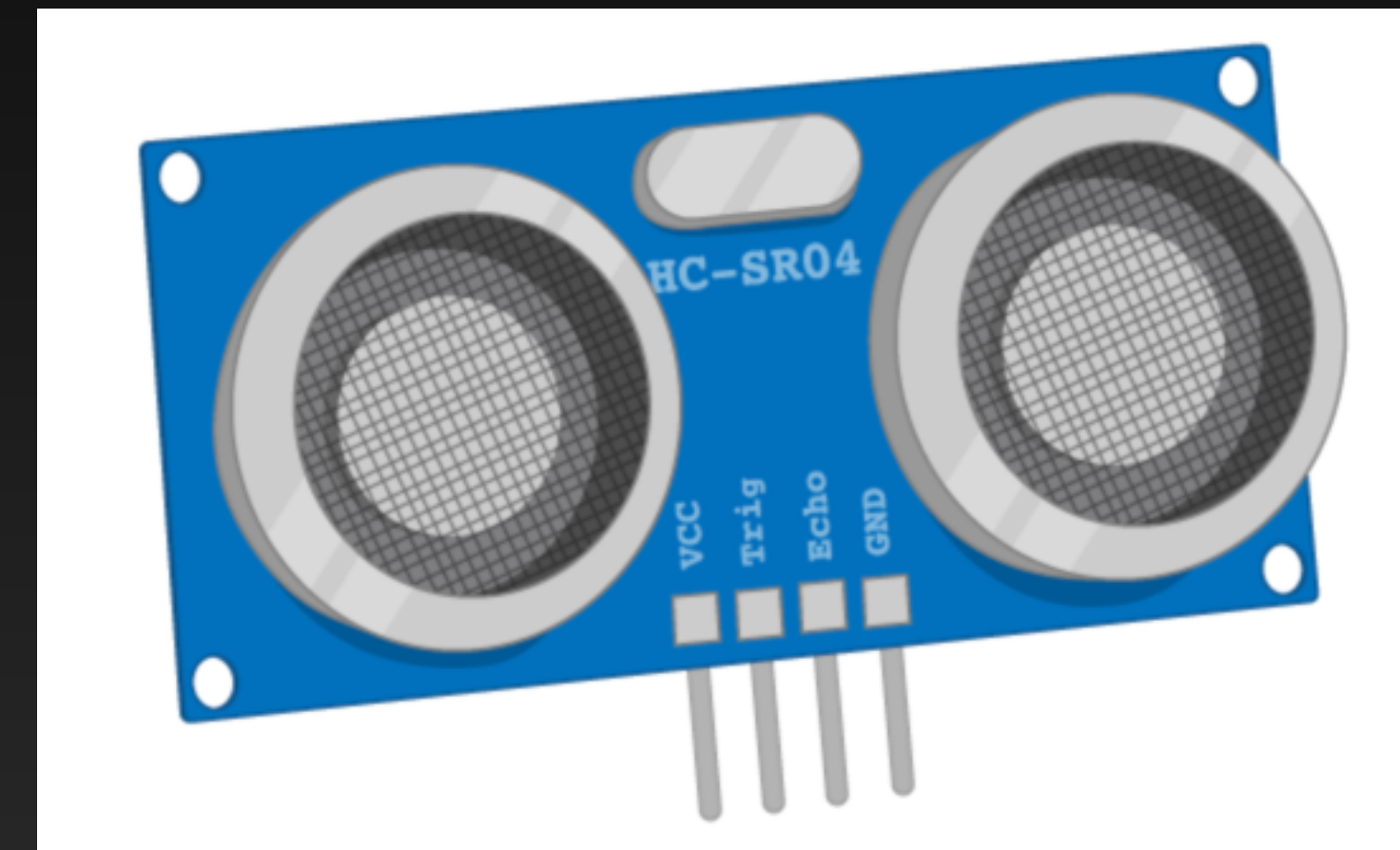
Setup Distance Sensor

An ultrasonic distance sensor is a device that sends out pulses of ultrasonic sound, and measures the time they take to bounce off nearby objects and be reflected back. They can measure distances fairly accurately (up to about a meter).

Setup Distance Sensor

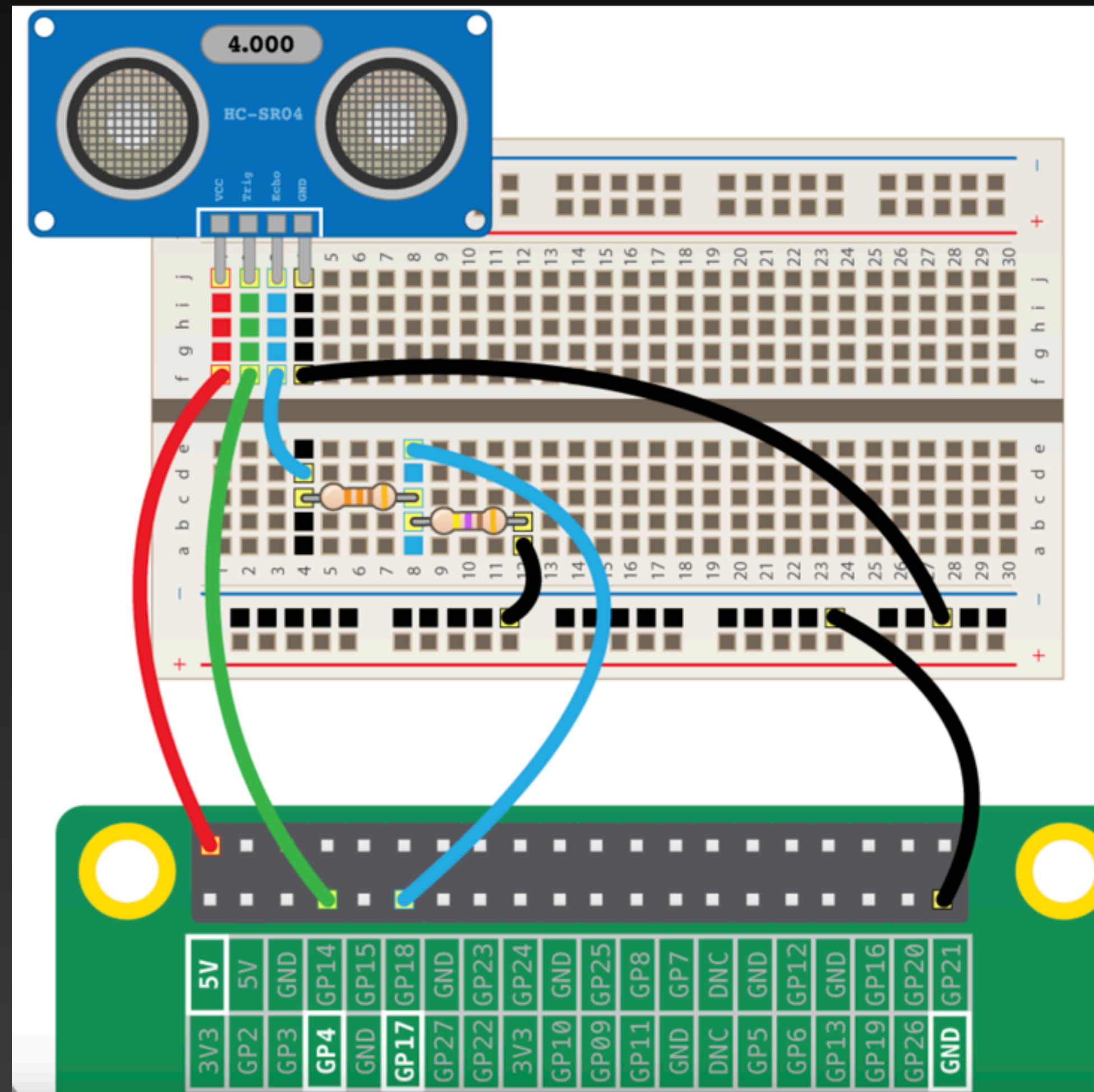
An ultrasonic distance sensor has four pins. They are called Ground (Gnd), Trigger (Trig), Echo (Echo), and Power (Vcc).

To use an ultrasonic distance sensor, you need to connect the Gnd pin to the ground pin on the Raspberry Pi, the Trig and Echo pins to GPIO pins on the Raspberry Pi, and the Vcc pin to the 3V3 pin on the Raspberry Pi.



Setup Distance Sensor

The circuit connects to two GPIO pins (one for echo, one for trigger), the ground pin, and a 5V pin. You'll need to use a pair of resistors (220Ω) as a potential divider:



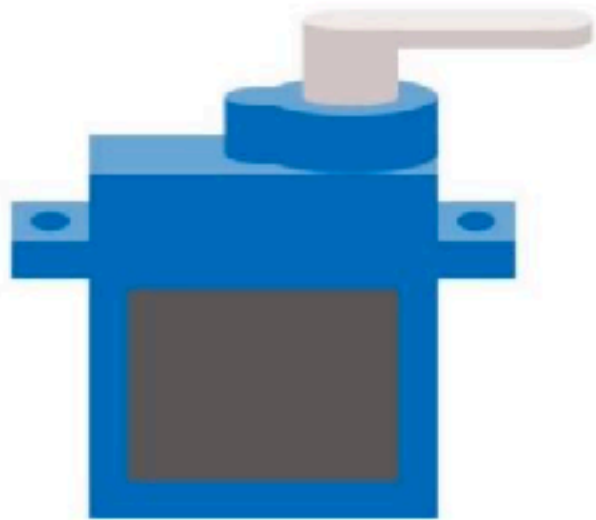
Setup Distance Sensor

```
from gpiozero import DistanceSensor
ultrasonic = DistanceSensor(echo=17, trigger=4)
while True:
    print(ultrasonic.distance)
```

Servo Motor Control

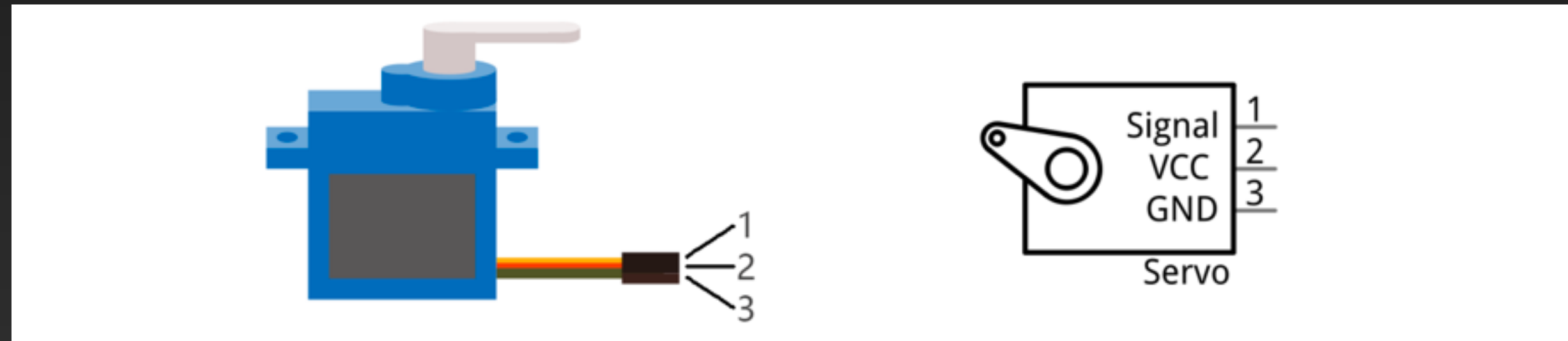
We will learn how to make motor rotate.

Component List

Raspberry Pi (with 40 GPIO) x1 GPIO Expansion Board & Ribbon Cable x1 Breadboard x1	Jumper Wire x3 
Servo x1 	

Servo Motor Control

Servo is a compact package which consists of a DC Motor, a set of reduction gears to provide torque, a sensor and control circuit board. Most Servos only have a 180-degree range of motion via their “horn”. Servos can output higher torque than a simple DC Motor alone and they are widely used to control motion in model cars, model airplanes, robots, etc. Servos have three wire leads which usually terminate to a male or female 3-pin plug. Two leads are for electric power: Positive (2-VCC, Red wire), Negative (3-GND, Brown wire), and the signal line (1-Signal, Orange wire) as represented in the Servo provided in your Kit.



Servo Motor Control

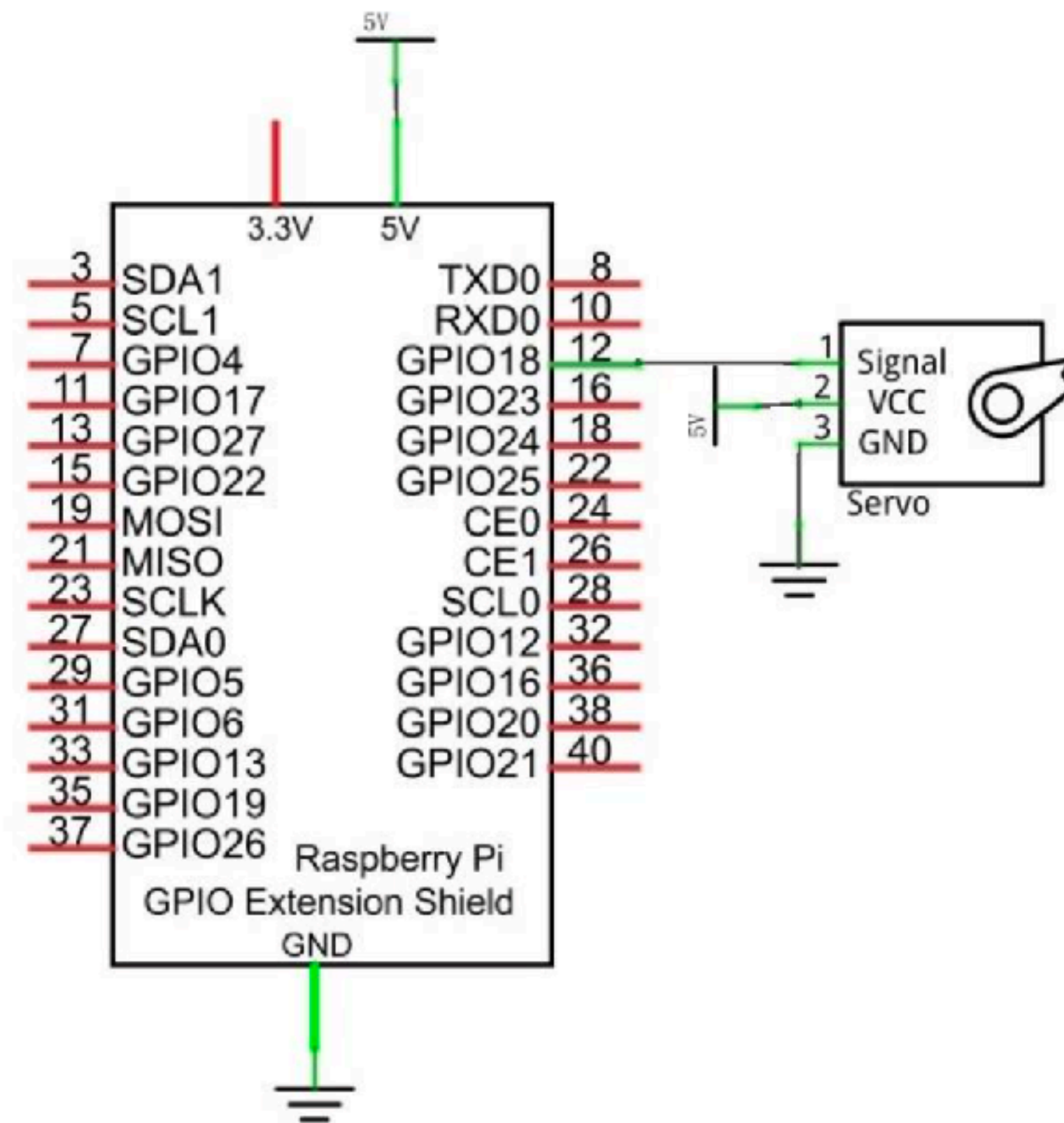
We will use a 50Hz PWM signal with a duty cycle in a certain range to drive the Servo. The lasting time 0.5ms- 2.5ms of PWM single cycle high level corresponds to the Servo angle 0 degrees - 180 degree linearly. Part of the corresponding values are as follows:

Note: the lasting time of high level corresponding to the servo angle is absolute instead of accumulating. For example, the high level time lasting for 0.5ms correspond to the 0 degree of the servo. If the high level time lasts for another 1ms, the servo rotates to 45 degrees.

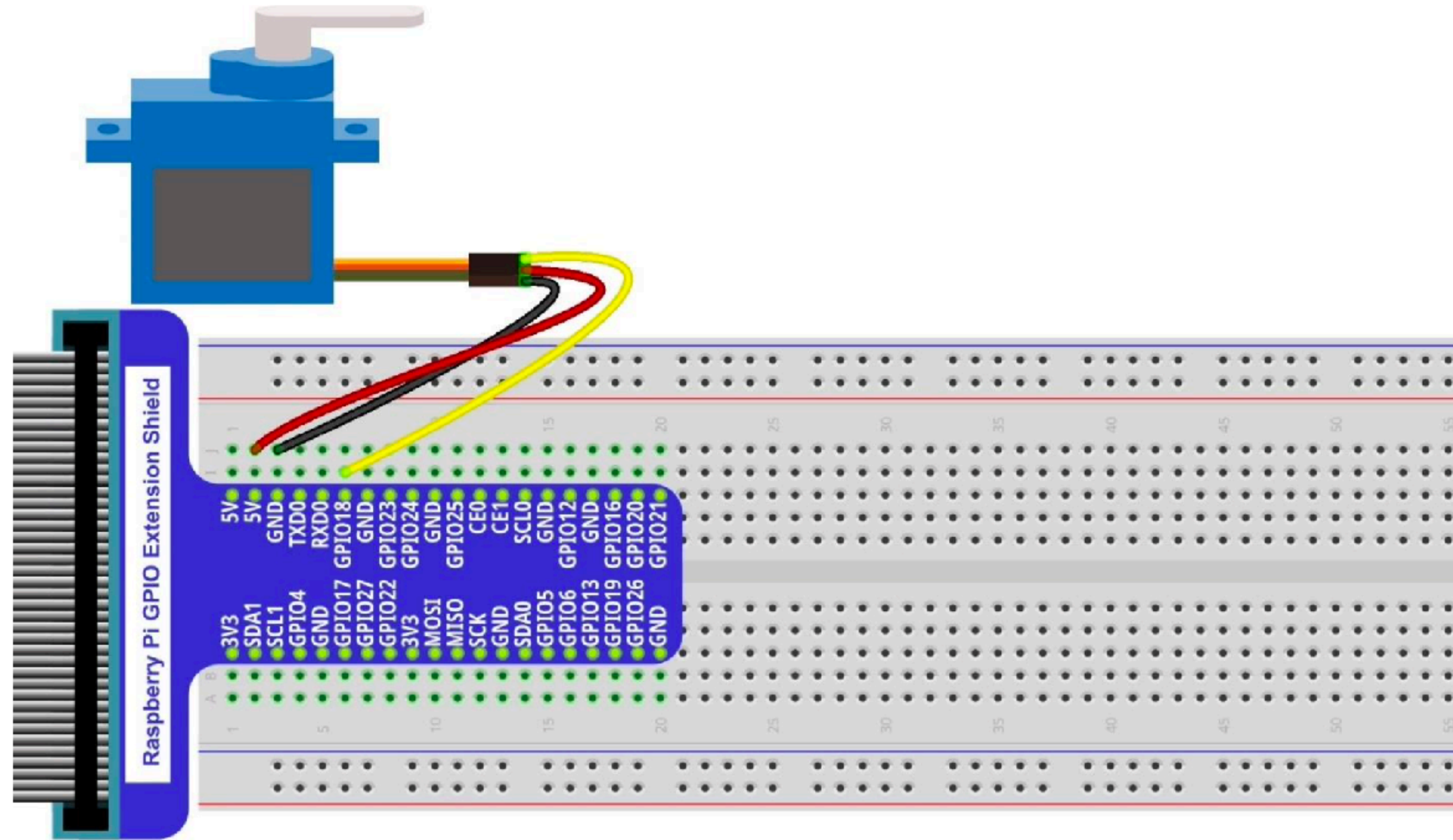
High level time	Servo angle
0.5ms	0 degree
1ms	45 degree
1.5ms	90 degree
2ms	135 degree
2.5ms	180 degree

Servo Motor Control

Schematic diagram



Servo Motor Control

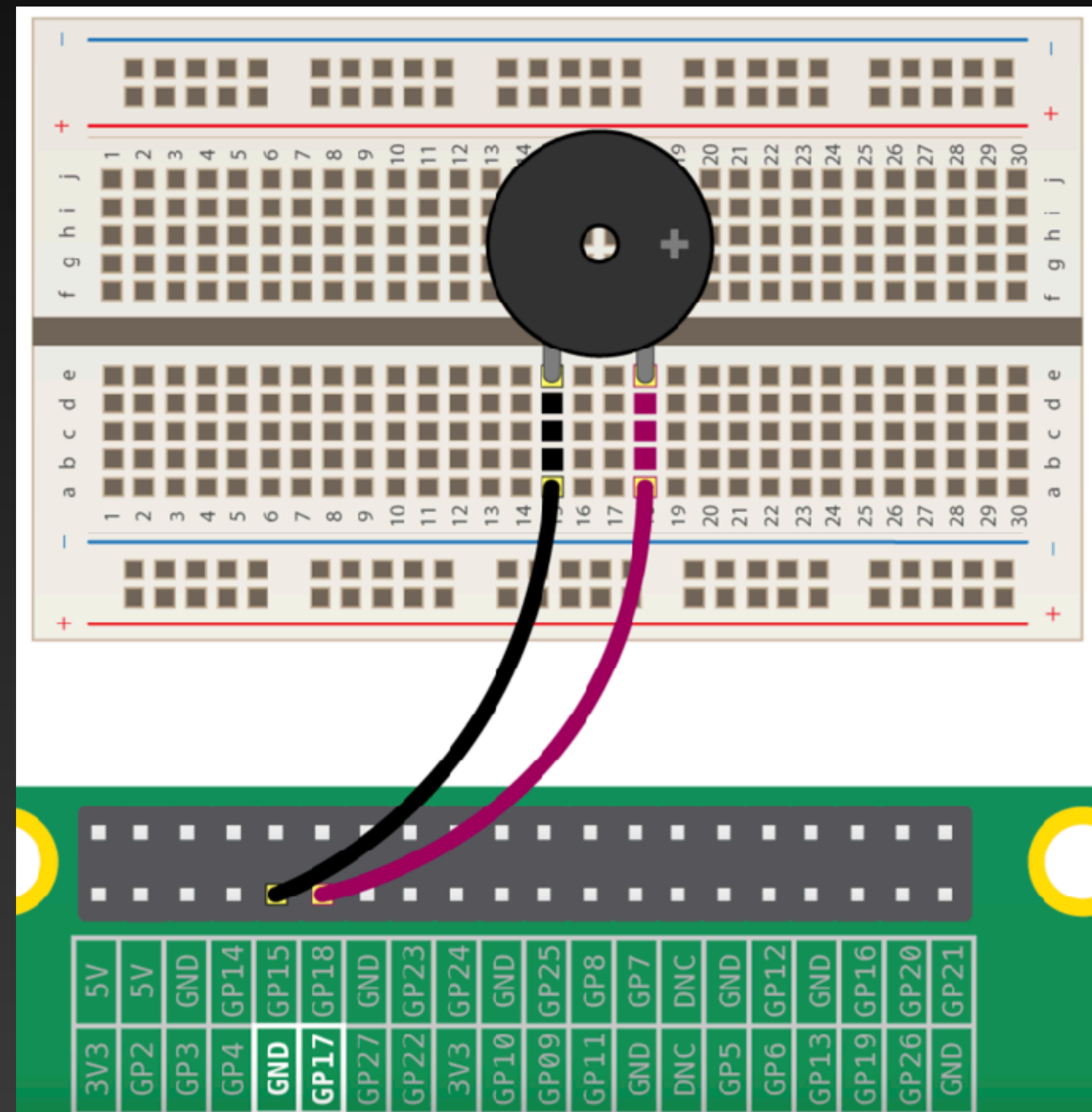


Servo Motor Control

```
pi@pi5:~/Freenove_Ultimate_Starter_Kit_for_Raspberry_Pi $ cd Code
pi@pi5:~/Freenove_Ultimate_Starter_Kit_for_Raspberry_Pi/Code $ cd Python_GPIOZero_Code/
pi@pi5:~/Freenove_Ultimate_Starter_Kit_for_Raspberry_Pi/Code/Python_GPIOZero_Code $ ls
00.0.0_Hello          06.2.1_Alertor          14.1.1_Relay          21.1.1_DHT11
01.1.1_Blink          07.1.1_ADC             15.1.1_Sweep          22.1.1_MatrixKeypad
02.1.1_ButtonLED     08.1.1_Softlight       16.1.1_SteppingMotor  23.1.1_SenseLED
02.2.1_Tablelamp     09.1.1_ColorfulSoftlight 17.1.1_LightWater02   24.1.1_UltrasonicRanging
03.1.1_LightWater    10.1.1_Nightlamp       18.1.1_SevenSegmentDisplay 25.1.1_MPU6050
04.1.1_BreathingLED  11.1.1_Thermometer     18.2.1_StopWatch     26.1.1_WebIO
05.1.1_ColorfulLED   12.1.1_Joystick        19.1.1_LEDMatrix     27.2.1_LightWater03
06.1.1_Doorbell      13.1.1_Motor           20.1.1_I2CLCD1602
pi@pi5:~/Freenove_Ultimate_Starter_Kit_for_Raspberry_Pi/Code/Python_GPIOZero_Code $ cd 15.1.1_Sweep/
pi@pi5:~/Freenove_Ultimate_Starter_Kit_for_Raspberry_Pi/Code/Python_GPIOZero_Code/15.1.1_Sweep $ python Sweep.py
/usr/lib/python3/dist-packages/gpiozero/output_devices.py:1509: PWMSoftwareFallback: To reduce servo jitter, use the pigpio pin factory. See https://gpiozero.readthedocs.io/en/stable/api_output.html#servo for more info
  warnings.warn(PWMSoftwareFallback(
Program is starting...
^Z
[1]+  Stopped                  python Sweep.py
pi@pi5:~/Freenove_Ultimate_Starter_Kit_for_Raspberry_Pi/Code/Python_GPIOZero_Code/15.1.1_Sweep $
```

Setup Buzzer

An active buzzer can be connected just like an LED, but as they are a little more robust, you won't be needing a resistor to protect them.



Setup Buzzer

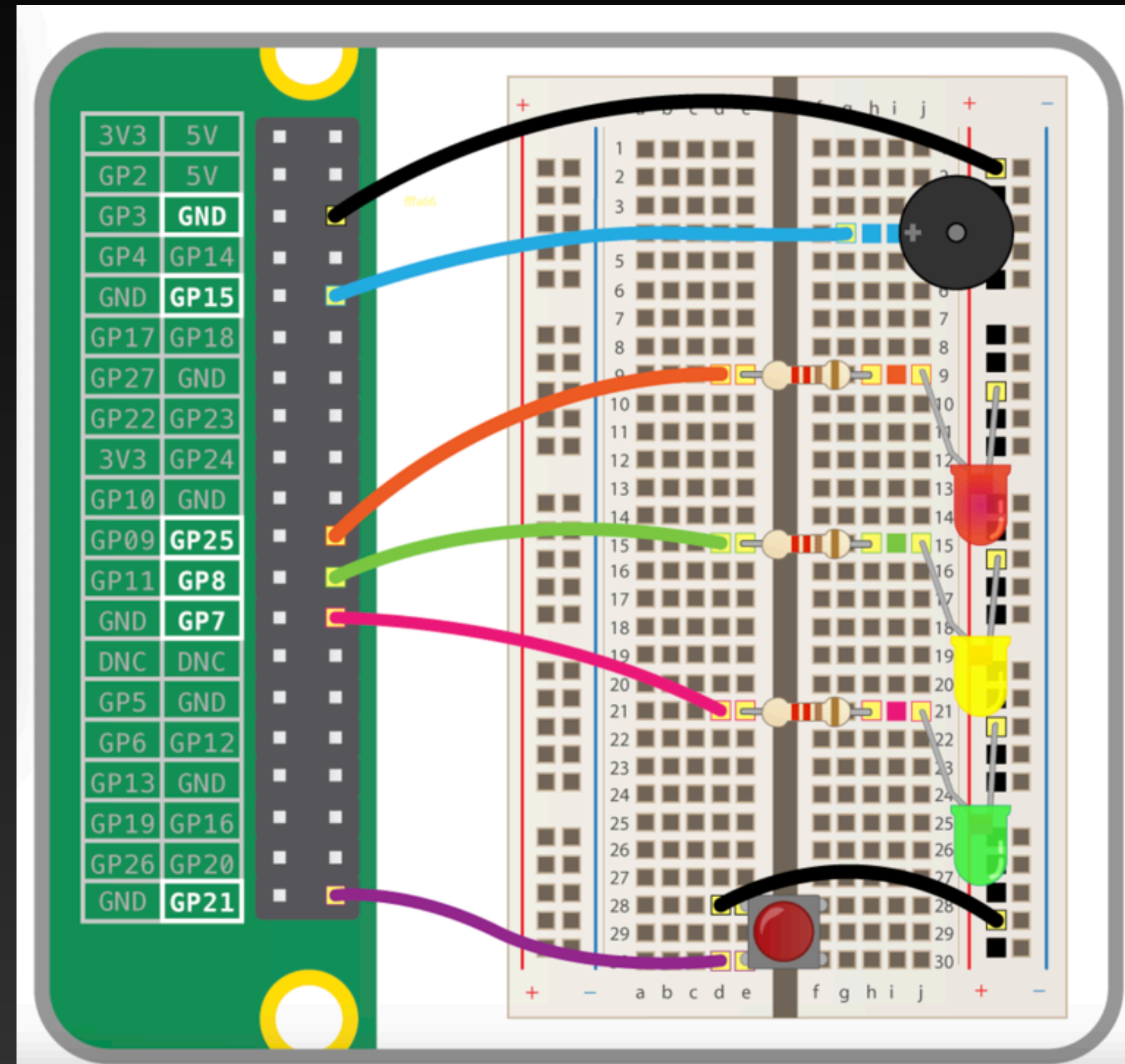
An active buzzer can be connected just like an LED, but as they are a little more robust, you won't be needing a resistor to protect them.

```
from gpiozero import Buzzer  
from time import sleep
```

```
buzzer = Buzzer(17)
```

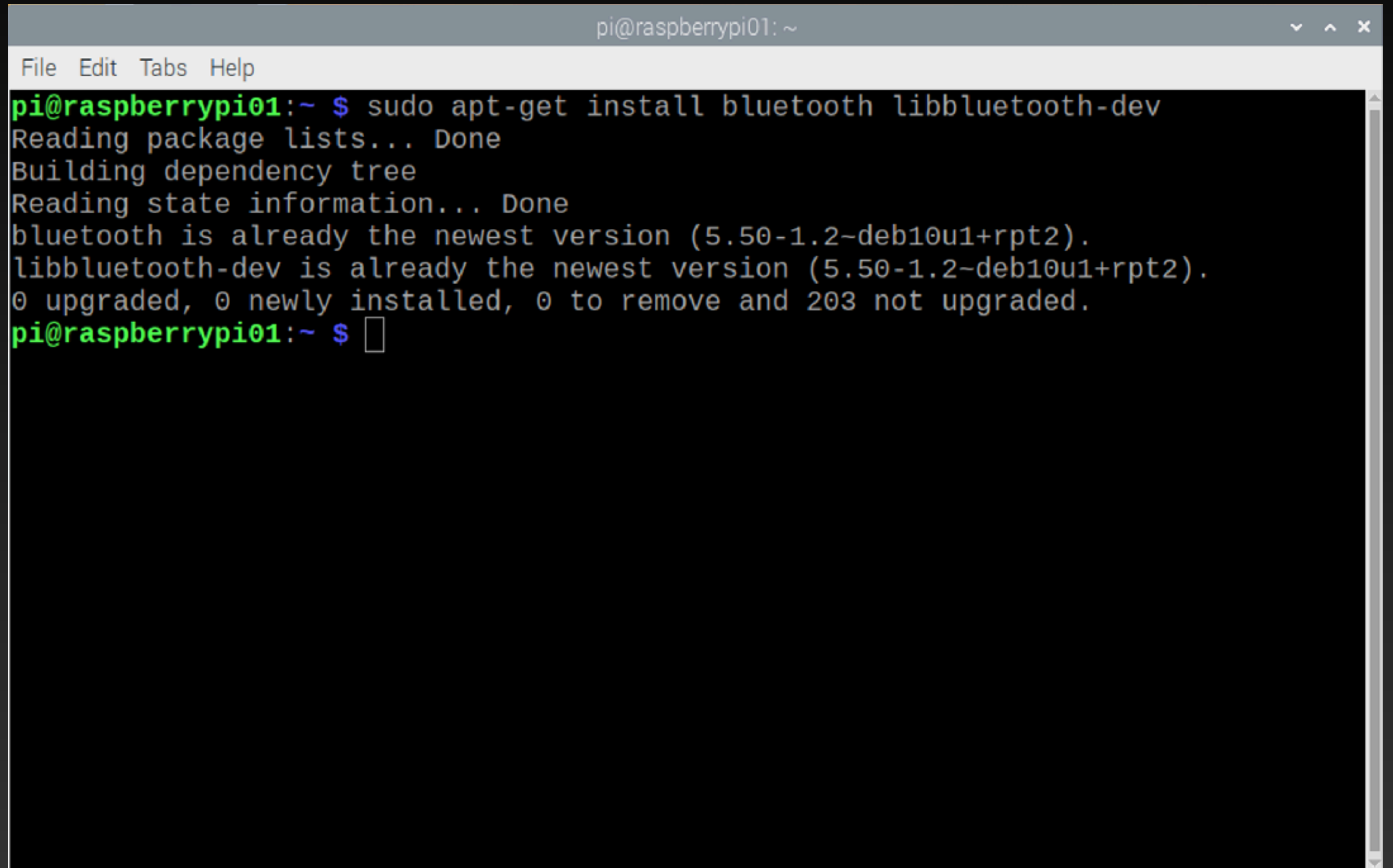
```
while True:  
    buzzer.on()  
    sleep(1)  
    buzzer.off()  
    sleep(1)
```


Setup Traffic light program



Python Program - Bluetooth setup

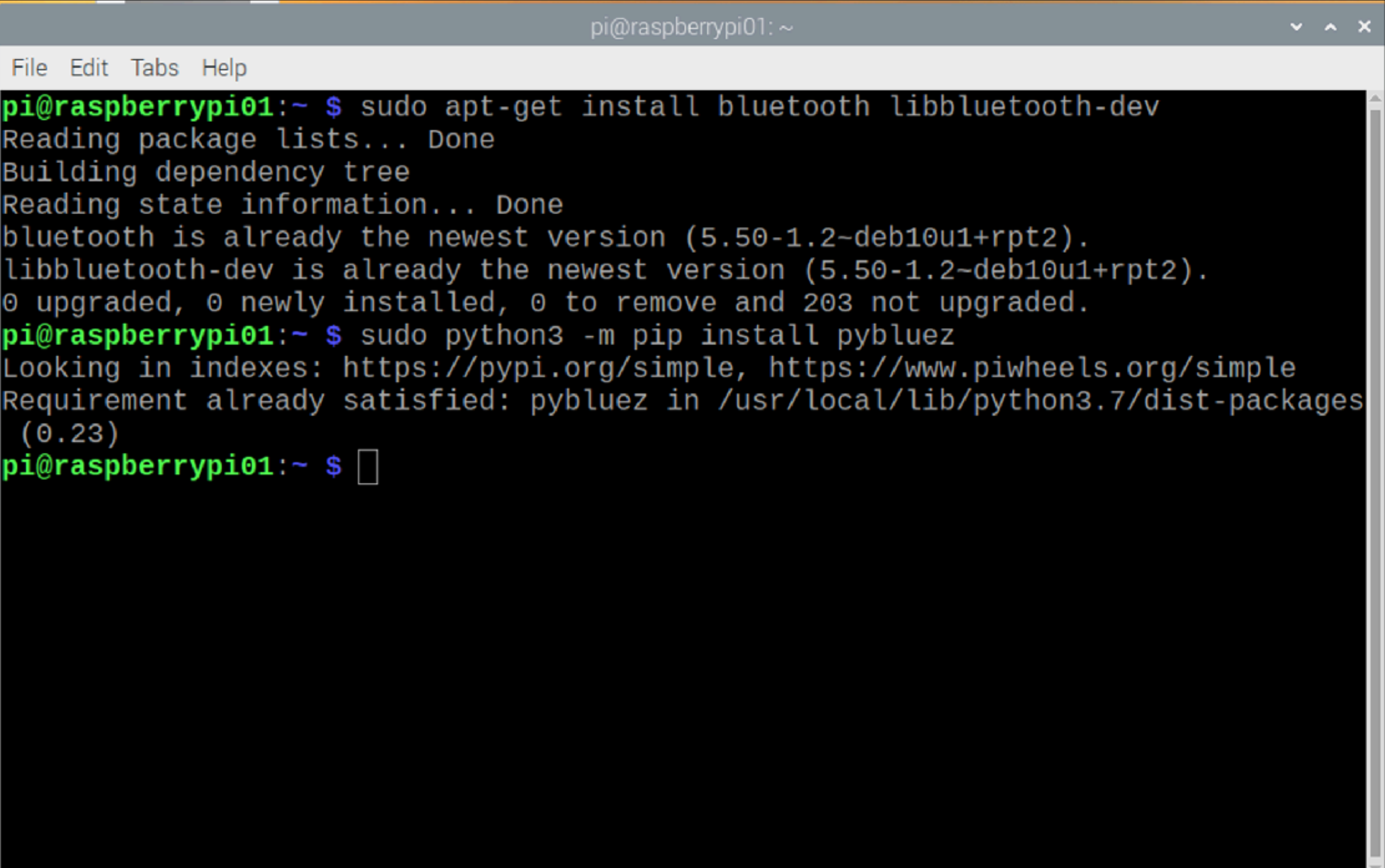
```
$ sudo apt-get install  
bluetooth libbluetooth-dev
```

A terminal window titled 'pi@raspberrypi01: ~' with a menu bar containing 'File', 'Edit', 'Tabs', and 'Help'. The terminal output shows the command 'sudo apt-get install bluetooth libbluetooth-dev' being executed. The output indicates that the packages are already the newest versions and no upgrades are needed.

```
pi@raspberrypi01:~ $ sudo apt-get install bluetooth libbluetooth-dev  
Reading package lists... Done  
Building dependency tree  
Reading state information... Done  
bluetooth is already the newest version (5.50-1.2-deb10u1+rpt2).  
libbluetooth-dev is already the newest version (5.50-1.2-deb10u1+rpt2).  
0 upgraded, 0 newly installed, 0 to remove and 203 not upgraded.  
pi@raspberrypi01:~ $
```

Python Program - Bluetooth setup

```
$ sudo python3 -m pip  
install pybluez
```



```
pi@raspberrypi01: ~  
File Edit Tabs Help  
pi@raspberrypi01:~ $ sudo apt-get install bluetooth libbluetooth-dev  
Reading package lists... Done  
Building dependency tree  
Reading state information... Done  
bluetooth is already the newest version (5.50-1.2-deb10u1+rpt2).  
libbluetooth-dev is already the newest version (5.50-1.2-deb10u1+rpt2).  
0 upgraded, 0 newly installed, 0 to remove and 203 not upgraded.  
pi@raspberrypi01:~ $ sudo python3 -m pip install pybluez  
Looking in indexes: https://pypi.org/simple, https://www.piwheels.org/simple  
Requirement already satisfied: pybluez in /usr/local/lib/python3.7/dist-packages  
(0.23)  
pi@raspberrypi01:~ $
```

Python Program - Bluetooth setup

```
$ sudo nano /etc/systemd/  
system/dbus-  
org.bluez.service
```

```
pi@raspberrypi01: ~  
File Edit Tabs Help  
GNU nano 3.2 /etc/systemd/system/dbus-org.bluez.service  
[Unit]  
Description=Bluetooth service  
Documentation=man:bluetoothd(8)  
ConditionPathIsDirectory=/sys/class/bluetooth  
  
[Service]  
Type=dbus  
BusName=org.bluez  
ExecStart=/usr/lib/bluetooth/bluetoothd -C  
ExecStartPost=/usr/bin/sdptool add SP  
NotifyAccess=main  
#WatchdogSec=10  
#Restart=on-failure  
CapabilityBoundingSet=CAP_NET_ADMIN CAP_NET_BIND_SERVICE  
LimitNPROC=1  
ProtectHome=true  
ProtectSystem=full  
  
[Install]  
[ Read 21 lines ]  
^G Get Help ^O Write Out ^W Where Is ^K Cut Text ^J Justify ^C Cur Pos  
^X Exit ^R Read File ^\ Replace ^U Uncut Text ^T To Spell ^_ Go To Line
```


Python Program - Bluetooth setup

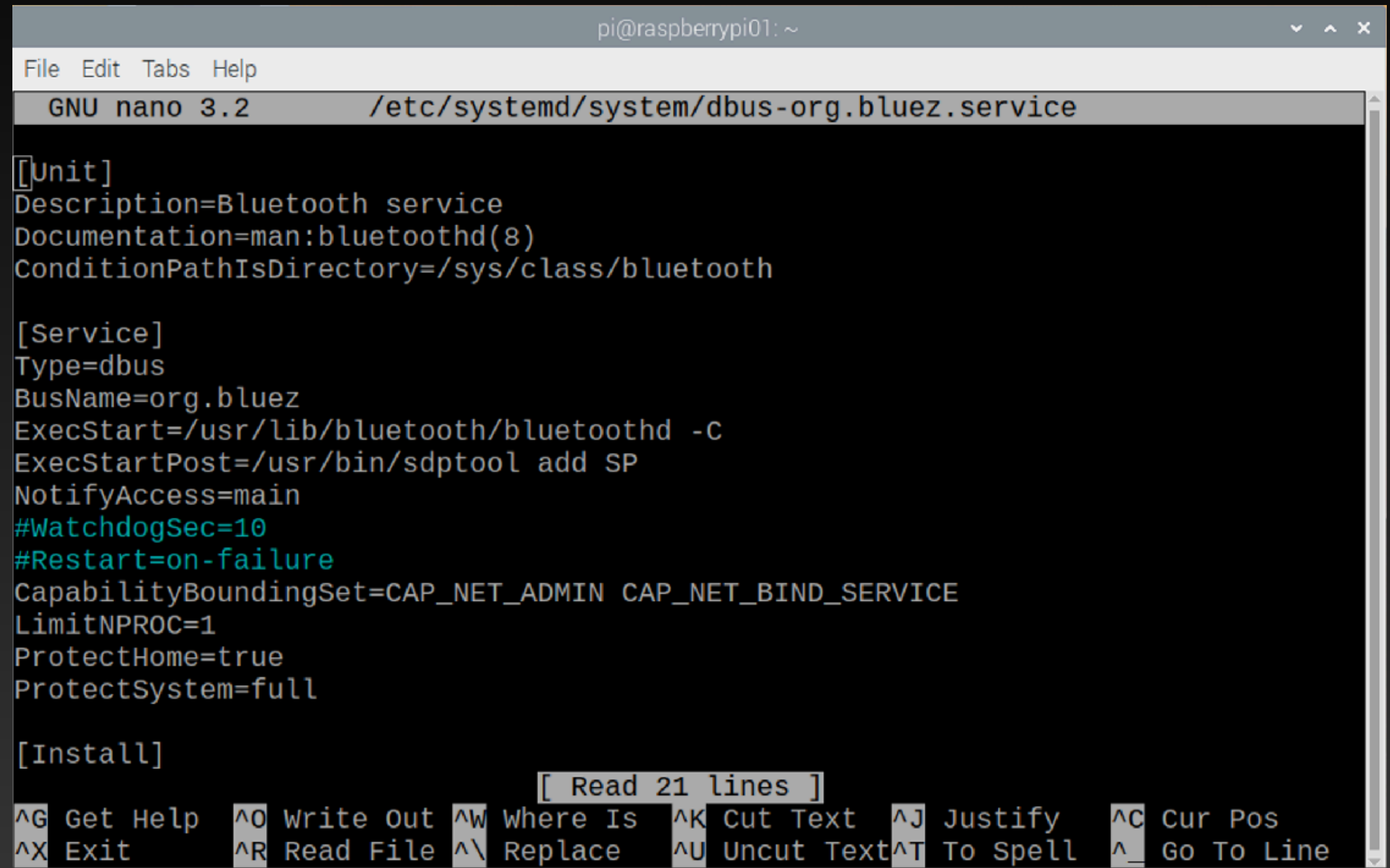
Add -C at the end of the ExecStart = line.

Also add another line after the ExecStart line.

The final two lines should look like below:

```
ExecStart=/usr/lib/  
bluetooth/bluetoothd -C
```

```
ExecStartPost=/usr/bin/  
sdptool add SP
```

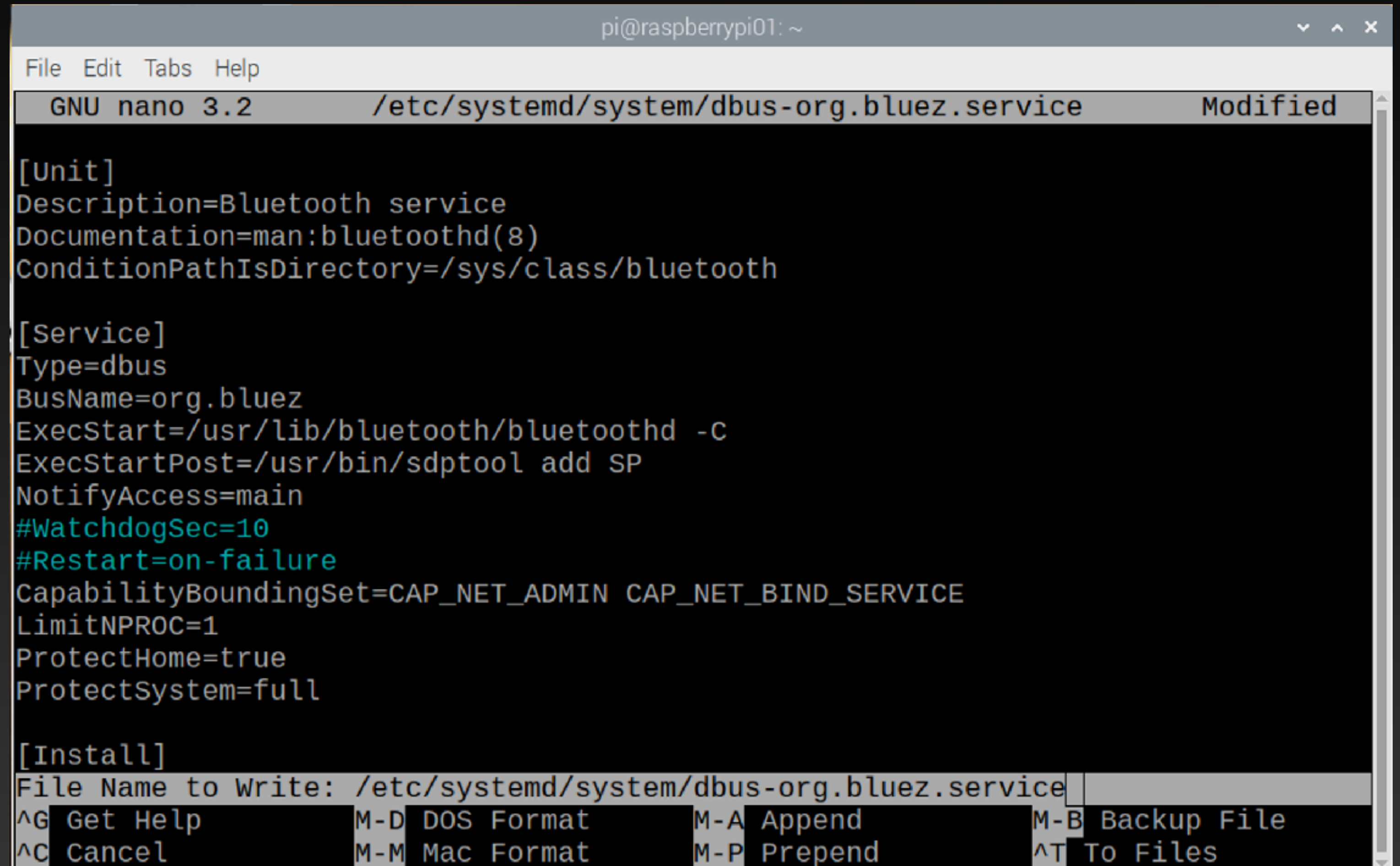


```
pi@raspberrypi01: ~  
File Edit Tabs Help  
GNU nano 3.2 /etc/systemd/system/dbus-org.bluez.service  
[Unit]  
Description=Bluetooth service  
Documentation=man:bluetoothd(8)  
ConditionPathIsDirectory=/sys/class/bluetooth  
  
[Service]  
Type=dbus  
BusName=org.bluez  
ExecStart=/usr/lib/bluetooth/bluetoothd -C  
ExecStartPost=/usr/bin/sdptool add SP  
NotifyAccess=main  
#WatchdogSec=10  
#Restart=on-failure  
CapabilityBoundingSet=CAP_NET_ADMIN CAP_NET_BIND_SERVICE  
LimitNPROC=1  
ProtectHome=true  
ProtectSystem=full  
  
[Install]  
[ Read 21 lines ]  
^G Get Help ^O Write Out ^W Where Is ^K Cut Text ^J Justify ^C Cur Pos  
^X Exit ^R Read File ^\ Replace ^U Uncut Text ^T To Spell ^_ Go To Line
```


Python Program - Bluetooth setup

Exit and save the file by entering Ctrl + X, followed by Y.

Press enter after type in Y, you will be back to terminal screen



```
pi@raspberrypi01: ~
File Edit Tabs Help
GNU nano 3.2 /etc/systemd/system/dbus-org.bluez.service Modified

[Unit]
Description=Bluetooth service
Documentation=man:bluetoothd(8)
ConditionPathIsDirectory=/sys/class/bluetooth

[Service]
Type=dbus
BusName=org.bluez
ExecStart=/usr/lib/bluetooth/bluetoothd -C
ExecStartPost=/usr/bin/sdptool add SP
NotifyAccess=main
#WatchdogSec=10
#Restart=on-failure
CapabilityBoundingSet=CAP_NET_ADMIN CAP_NET_BIND_SERVICE
LimitNPROC=1
ProtectHome=true
ProtectSystem=full

[Install]
File Name to Write: /etc/systemd/system/dbus-org.bluez.service
^G Get Help      M-D DOS Format   M-A Append      M-B Backup File
^C Cancel        M-M Mac Format   M-P Prepend     ^T To Files
```

Python Program - Bluetooth setup

\$ sudo reboot

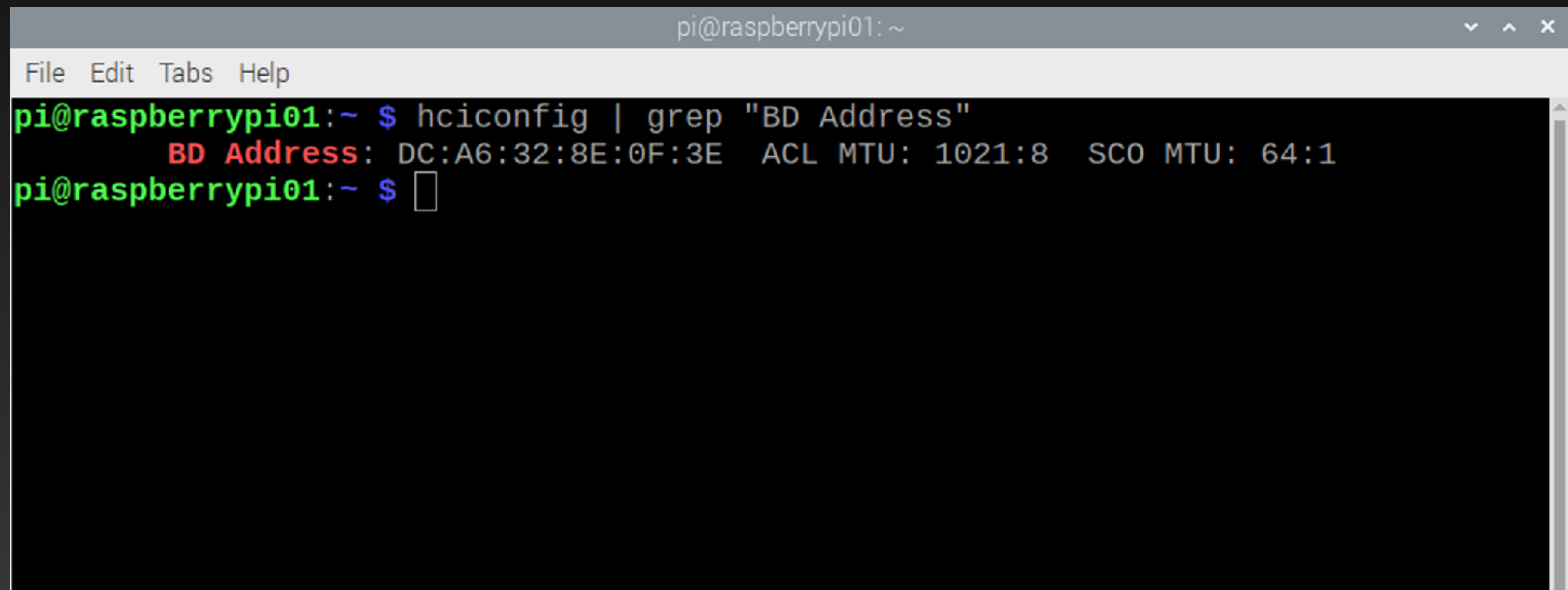
Press enter, your
Raspberry Pi will
reboot

```
pi@raspberrypi01: ~  
File Edit Tabs Help  
pi@raspberrypi01:~ $ sudo apt-get install bluetooth libbluetooth-dev  
Reading package lists... Done  
Building dependency tree  
Reading state information... Done  
bluetooth is already the newest version (5.50-1.2-deb10u1+rpt2).  
libbluetooth-dev is already the newest version (5.50-1.2-deb10u1+rpt2).  
0 upgraded, 0 newly installed, 0 to remove and 203 not upgraded.  
pi@raspberrypi01:~ $ sudo python3 -m pip install pybluez  
Looking in indexes: https://pypi.org/simple, https://www.piwheels.org/simple  
Requirement already satisfied: pybluez in /usr/local/lib/python3.7/dist-packages  
(0.23)  
pi@raspberrypi01:~ $ sudo nano /etc/systemd/system/dbus-org.bluez.service  
pi@raspberrypi01:~ $ sudo nano /etc/systemd/system/dbus-org.bluez.service  
pi@raspberrypi01:~ $ sudo reboot
```

Bluetooth Pairing with Android Devices

Open your terminal, type below command:

```
$ hciconfig | grep "BD Address"
```



```
pi@raspberrypi01: ~  
File Edit Tabs Help  
pi@raspberrypi01:~ $ hciconfig | grep "BD Address"  
    BD Address: DC:A6:32:8E:0F:3E  ACL MTU: 1021:8  SCO MTU: 64:1  
pi@raspberrypi01:~ $
```

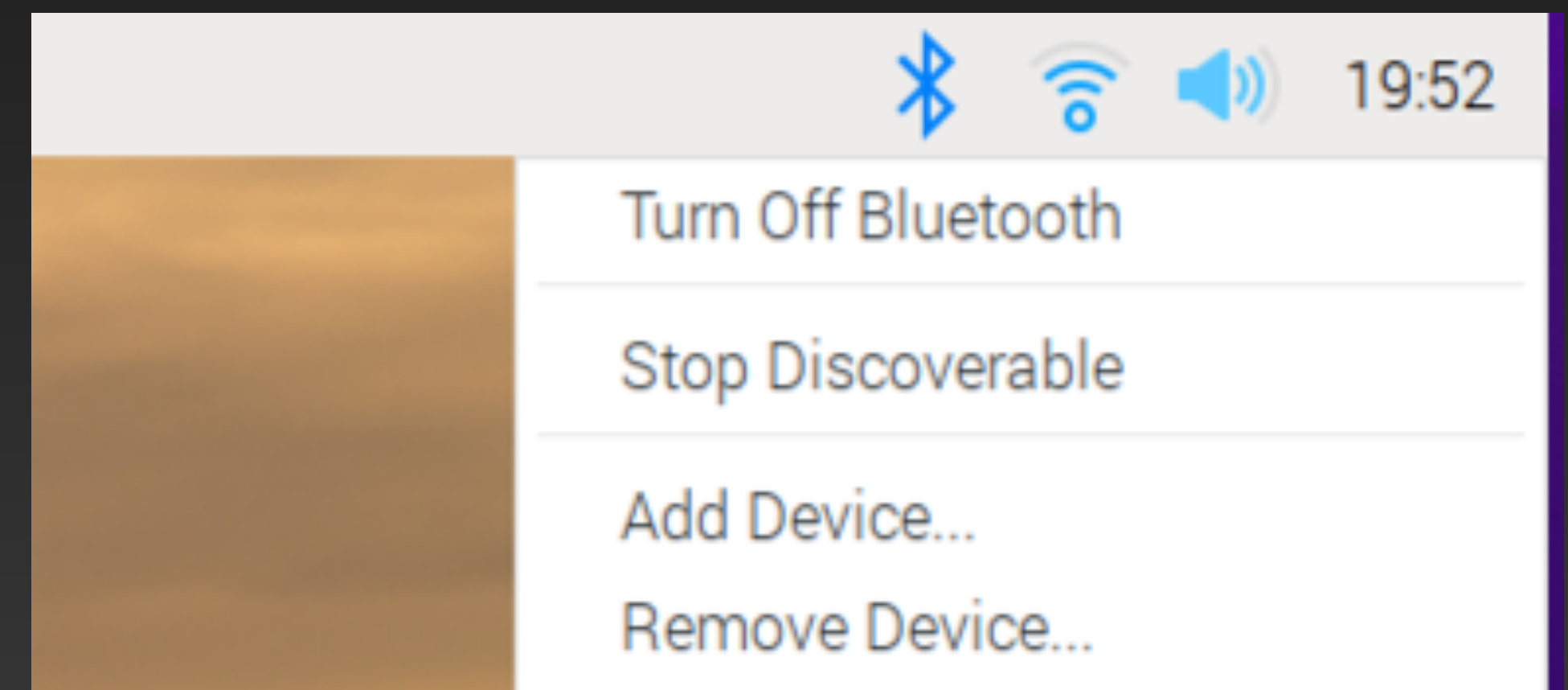
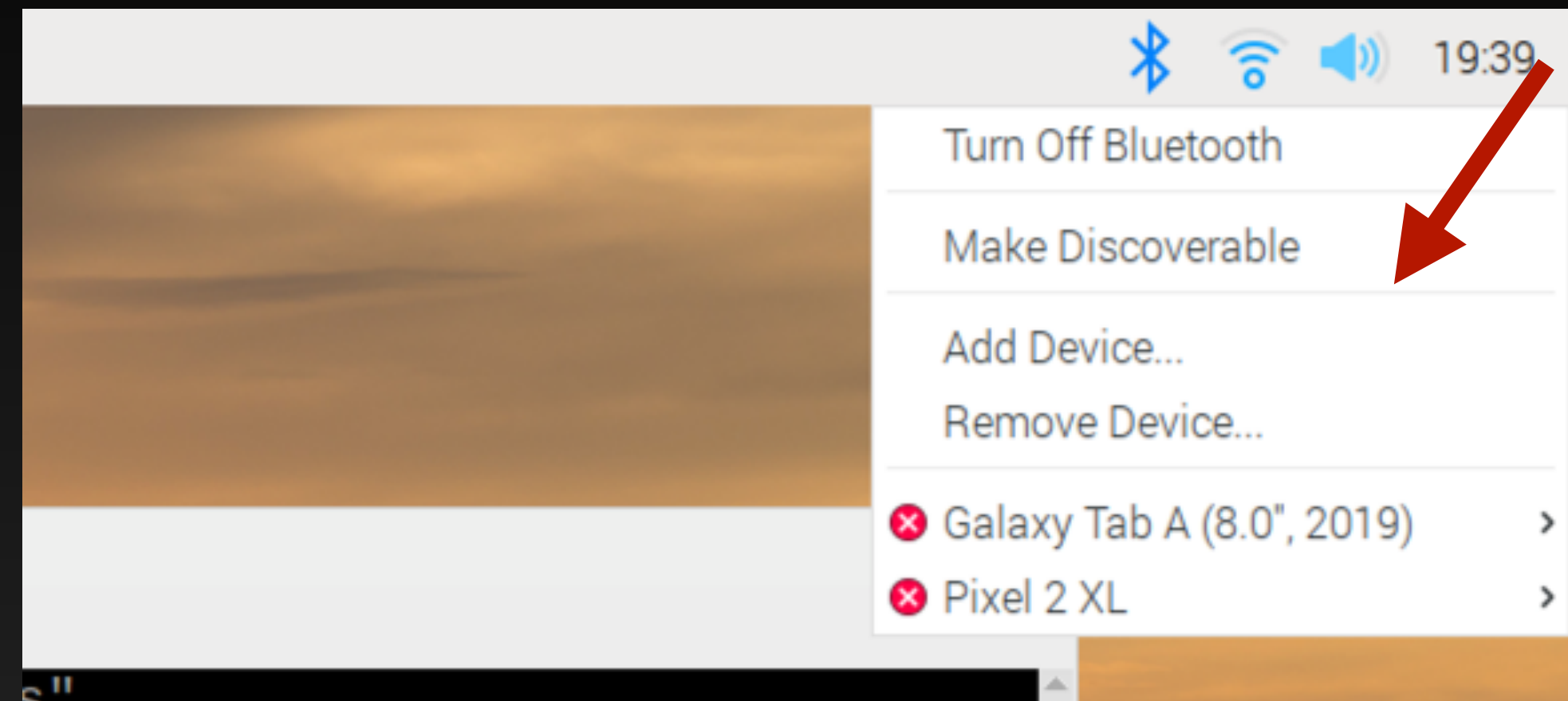

Bluetooth Pairing with Android Devices

MAC address: A media access control address (MAC address) is a unique identifier assigned to a network interface controller (NIC) for use as a network address in communications within a network segment. This use is common in most IEEE 802 networking technologies, including Ethernet, Wi-Fi, and Bluetooth.

Bluetooth Pairing with Android Devices

Config your raspberry Pi to discoverable.

First make sure the bluetooth is turned on, then click on Make Discoverable, you should see the bluetooth icon is flashing in green and blue color.



Bluetooth Pairing with Android Devices

Go to your Android device, find connection under settings.

Pair raspberry pi with your device.



Bluetooth Pairing with Android Devices

Debug from terminal.

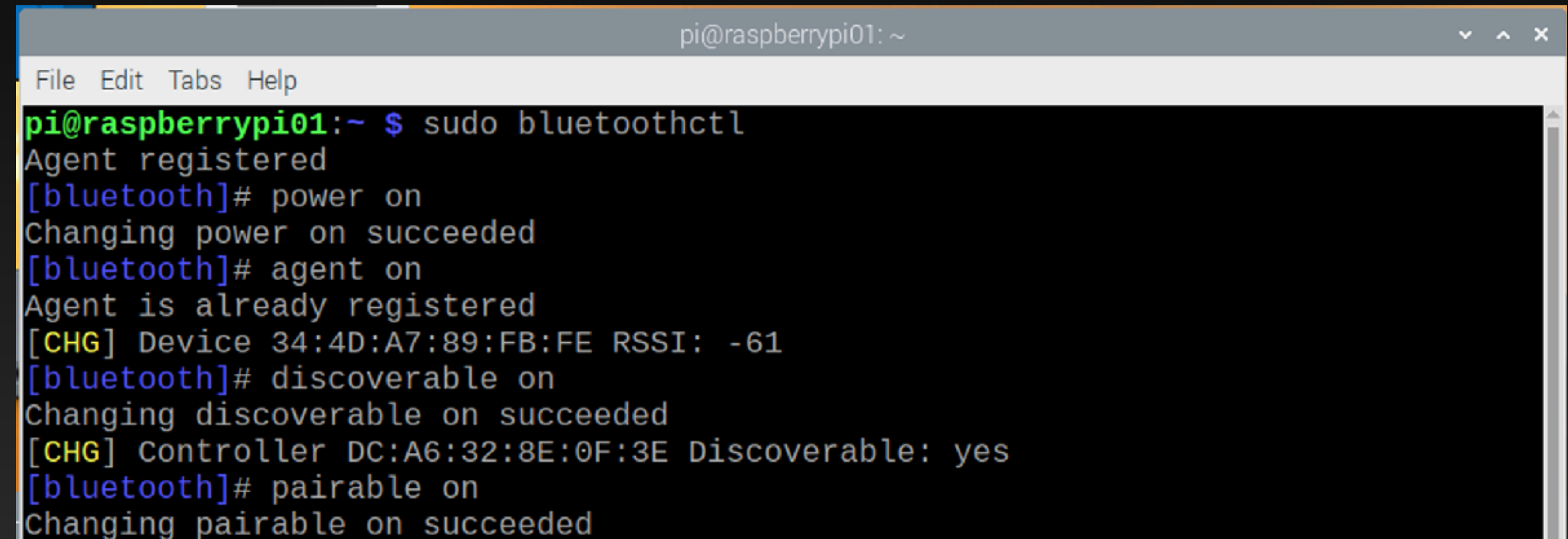
```
$ sudo bluetoothctl
```

```
# power on
```

```
# agent on
```

```
# discoverable on
```

```
# pairable on
```

A screenshot of a terminal window titled 'pi@raspberrypi01: ~'. The terminal shows the execution of the 'sudo bluetoothctl' command. The output shows the Bluetooth agent being registered and several settings being turned on: power, agent, discoverable, and pairable. The terminal text is as follows:

```
pi@raspberrypi01:~ $ sudo bluetoothctl
Agent registered
[bluetooth]# power on
Changing power on succeeded
[bluetooth]# agent on
Agent is already registered
[CHG] Device 34:4D:A7:89:FB:FE RSSI: -61
[bluetooth]# discoverable on
Changing discoverable on succeeded
[CHG] Controller DC:A6:32:8E:0F:3E Discoverable: yes
[bluetooth]# pairable on
Changing pairable on succeeded
```


Bluetooth Pairing with Android Devices

```
# scan on
```

After the last command “*scan on*”, you will see your Bluetooth device (Mobile phone) in the list. Make sure that your mobile has Bluetooth turned on and visible by nearby devices. Then copy the MAC address of you device and pair it by using given command:

```
# pair <address of your phone>
```

```
pi@raspberrypi01: ~  
File Edit Tabs Help  
[CHG] Device 34:4D:A7:89:FB:FE RSSI: -48  
[CHG] Device 67:FE:8A:0B:AD:3B ManufacturerData Key: 0x004c  
[CHG] Device 67:FE:8A:0B:AD:3B ManufacturerData Value:  
0c 0e 08 90 93 63 d5 90 9b 65 ec a6 ac 00 f0 0a .....c...e.....  
10 06 4b 1d 33 95 0b 58 ..K.3..X  
[CHG] Device D0:03:DF:D2:BC:25 ManufacturerData Key: 0x0075  
[CHG] Device D0:03:DF:D2:BC:25 ManufacturerData Value:  
42 04 01 01 66 d0 03 df d2 bc 25 d2 03 df d2 bc B...f.....%.....  
24 01 ba 00 00 00 00 00 $.....  
[CHG] Device 67:FE:8A:0B:AD:3B RSSI: -97  
[bluetooth]# scan on  
Discovery started  
[CHG] Device 67:FE:8A:0B:AD:3B ManufacturerData Key: 0x004c  
[CHG] Device 67:FE:8A:0B:AD:3B ManufacturerData Value:  
0c 0e 08 91 93 27 de db dd bd f6 6a a2 14 66 f2 .....'......j..f..  
10 06 4b 1d 33 95 0b 58 ..K.3..X  
[CHG] Device 4E:74:66:4A:C8:C2 RSSI: -73  
[CHG] Device 46:3D:5F:EC:F3:CD RSSI: -68  
[CHG] Device D0:03:DF:D2:BC:25 ManufacturerData Key: 0x0075  
[CHG] Device D0:03:DF:D2:BC:25 ManufacturerData Value:  
42 04 01 20 66 19 0f 00 00 01 2b 00 00 00 00 00 B.. f.....+.....  
00 00 00 00 00 00 00 00 .....  
[NEW] Device 57:A7:BF:4A:65:53 57-A7-BF-4A-65-53  
[bluetooth]#
```


Python Program - LED control through bluetooth

Notice that what is returned into variable data is byte objects which is actually arrays of integers.

Here, we are using the built-in decode function to convert them into their equivalent ASCII characters.

In Python3, utf-8 is the default encoding and decoding parameter.

```
Decode MAC to ASCII (utf-8 in python 3)
#-----
#
#       decode Pi signal/data
#
#-----

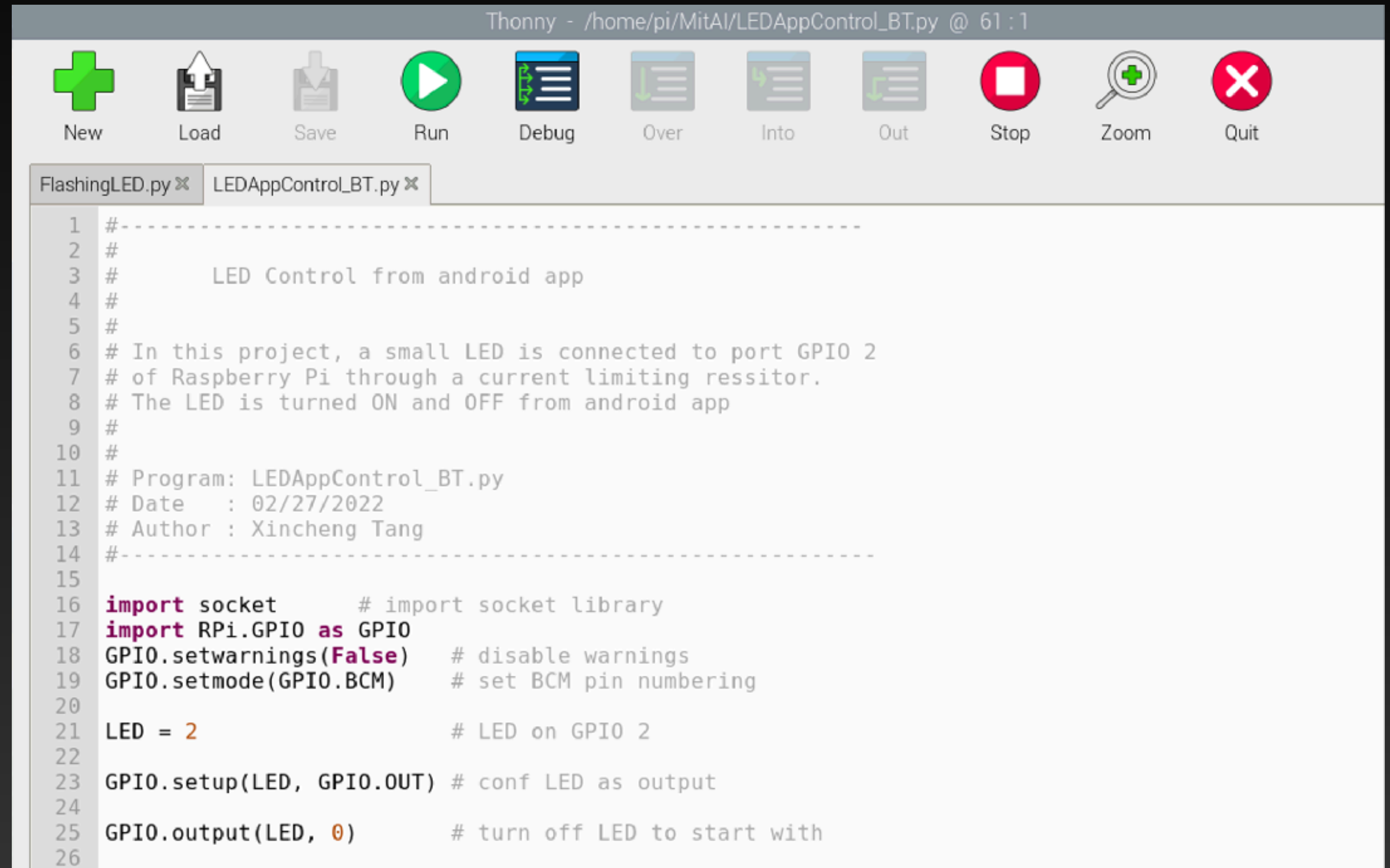
try:
    while True:
        data = client.recv(1024)           # Recieve data bytes
        if data.decode('utf-8') == '1':    # 1 received
            GPIO.output(LED, 1)            # LED ON
        elif data.decode('utf-8') == '0':  # 0 received
            GPIO.output(LED, 0)            # LED OFF

except KeyboardInterrupt:                  # Keyboard interrupt
    client.close()
    s.close()
```

Python Program - LED control through bluetooth

Develop the python program
to for bluetooth control.

GPIO library setup



```
Thonny - /home/pi/MitAI/LEDAppControl_BT.py @ 61 : 1

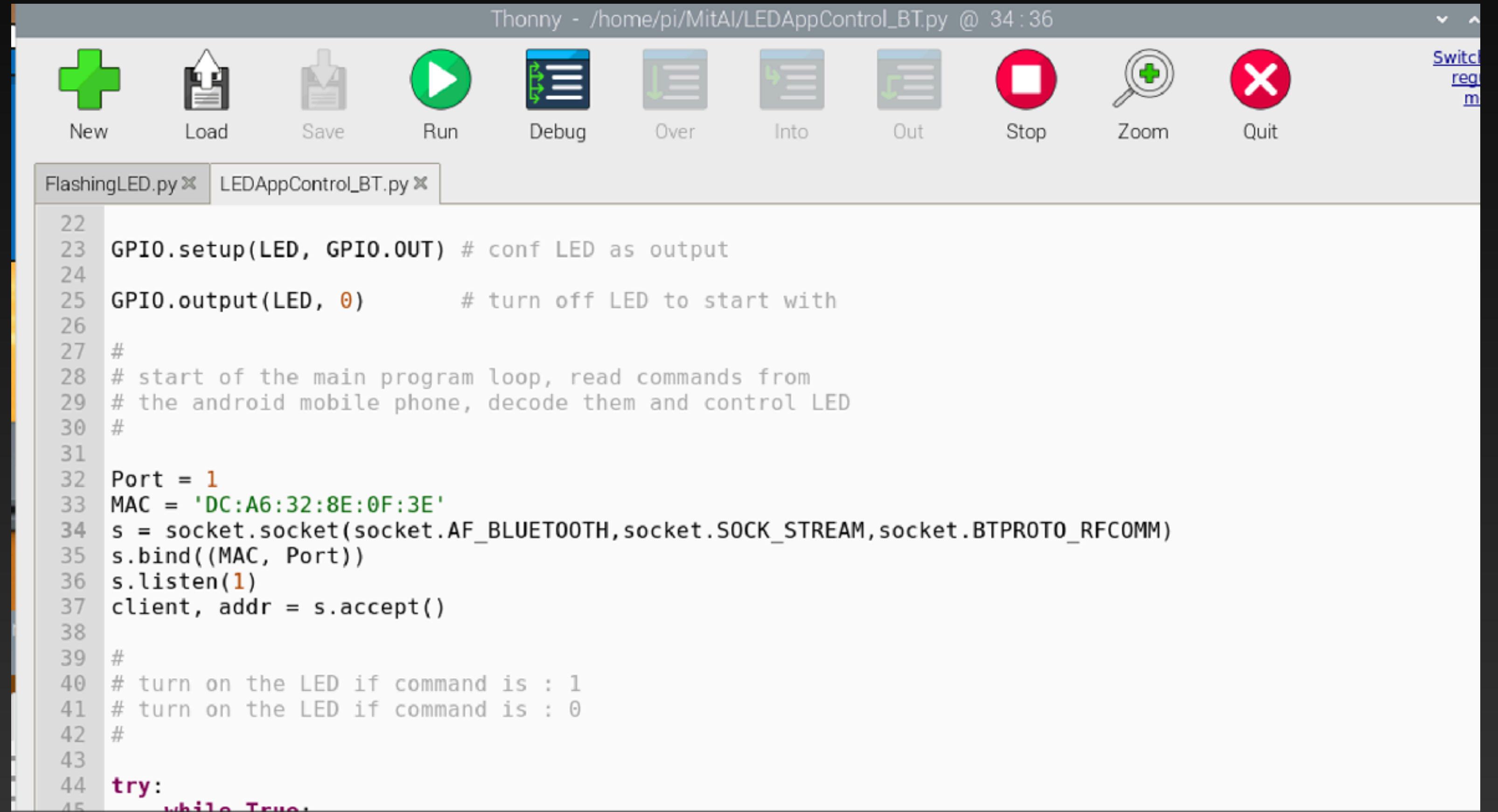
New Load Save Run Debug Over Into Out Stop Zoom Quit

FlashingLED.py LEDAppControl_BT.py

1 #-----
2 #
3 # LED Control from android app
4 #
5 #
6 # In this project, a small LED is connected to port GPIO 2
7 # of Raspberry Pi through a current limiting resistor.
8 # The LED is turned ON and OFF from android app
9 #
10 #
11 # Program: LEDAppControl_BT.py
12 # Date : 02/27/2022
13 # Author : Xincheng Tang
14 #-----
15
16 import socket # import socket library
17 import RPi.GPIO as GPIO
18 GPIO.setwarnings(False) # disable warnings
19 GPIO.setmode(GPIO.BCM) # set BCM pin numbering
20
21 LED = 2 # LED on GPIO 2
22
23 GPIO.setup(LED, GPIO.OUT) # conf LED as output
24
25 GPIO.output(LED, 0) # turn off LED to start with
26
```

Python Program - LED control through bluetooth

Setup socket to receive signal from physical device.

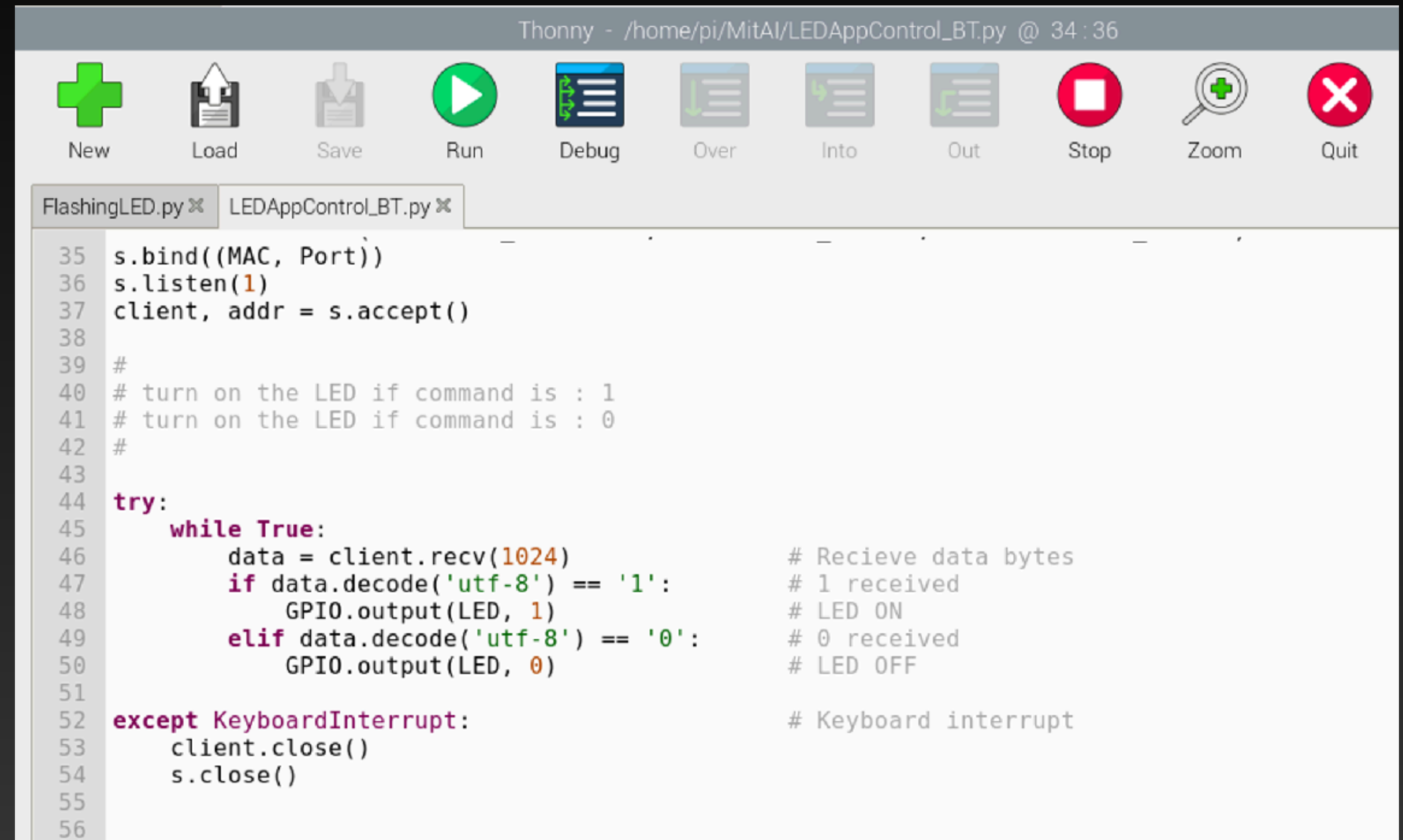


The screenshot shows the Thonny Python IDE interface. The title bar indicates the file path: Thonny - /home/pi/MitAI/LEDAppControl_BT.py @ 34 : 36. The toolbar contains icons for New, Load, Save, Run, Debug, Over, Into, Out, Stop, Zoom, and Quit. Below the toolbar, two tabs are visible: 'FlashingLED.py' and 'LEDAppControl_BT.py'. The 'LEDAppControl_BT.py' tab is active, displaying the following Python code:

```
22
23 GPIO.setup(LED, GPIO.OUT) # conf LED as output
24
25 GPIO.output(LED, 0)      # turn off LED to start with
26
27 #
28 # start of the main program loop, read commands from
29 # the android mobile phone, decode them and control LED
30 #
31
32 Port = 1
33 MAC = 'DC:A6:32:8E:0F:3E'
34 s = socket.socket(socket.AF_BLUETOOTH, socket.SOCK_STREAM, socket.BTPROTO_RFCOMM)
35 s.bind((MAC, Port))
36 s.listen(1)
37 client, addr = s.accept()
38
39 #
40 # turn on the LED if command is : 1
41 # turn on the LED if command is : 0
42 #
43
44 try:
45     while True:
```


Python Program - LED control through bluetooth

Decode the signal.

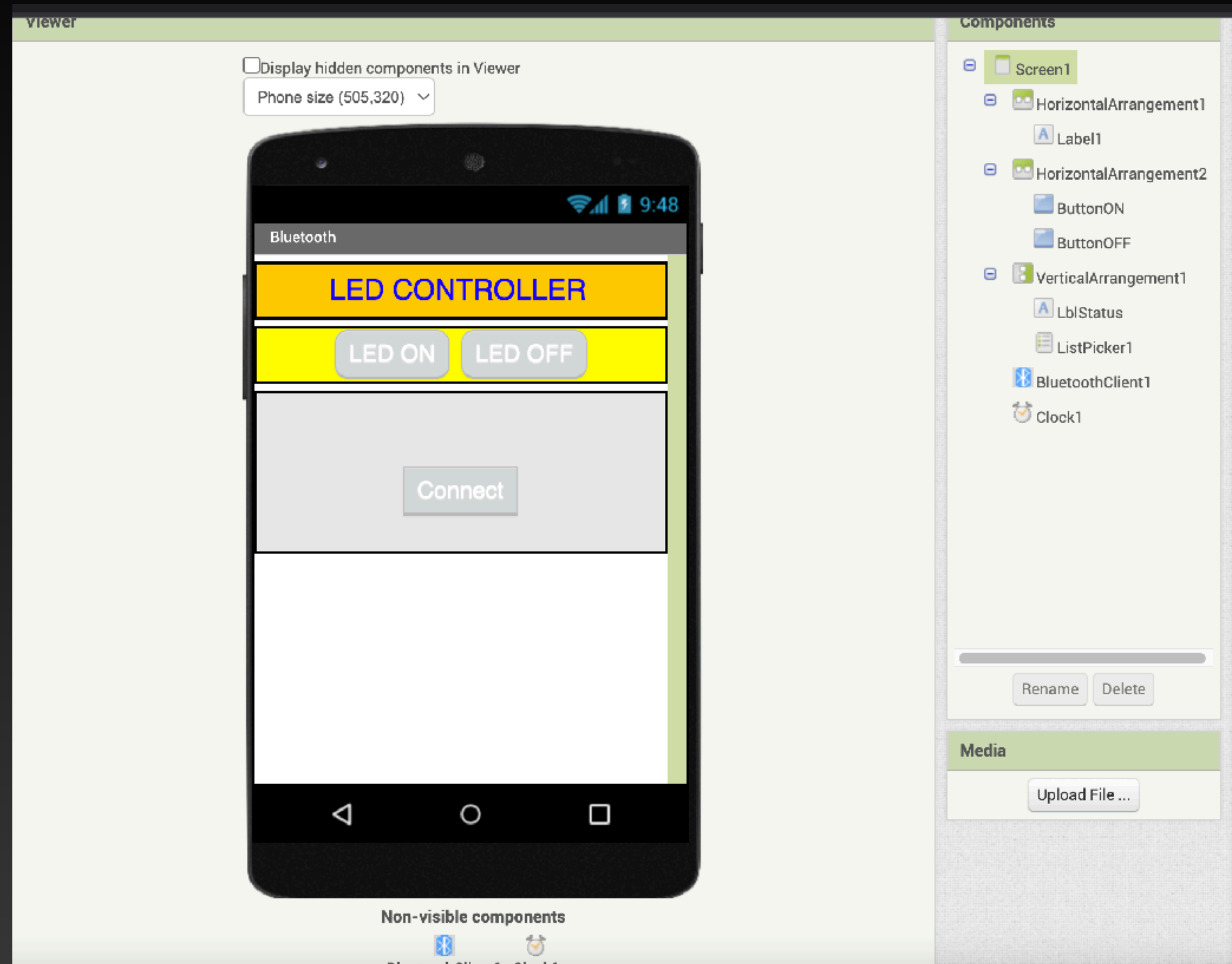


The screenshot shows the Thonny Python IDE interface. The title bar indicates the file path: Thonny - /home/pi/MitAI/LEDAppControl_BT.py @ 34:36. The toolbar contains icons for New, Load, Save, Run, Debug, Over, Into, Out, Stop, Zoom, and Quit. The file explorer shows two open files: FlashingLED.py and LEDAppControl_BT.py. The main editor displays the following Python code:

```
35 s.bind((MAC, Port))
36 s.listen(1)
37 client, addr = s.accept()
38
39 #
40 # turn on the LED if command is : 1
41 # turn on the LED if command is : 0
42 #
43
44 try:
45     while True:
46         data = client.recv(1024)           # Recieve data bytes
47         if data.decode('utf-8') == '1':    # 1 received
48             GPIO.output(LED, 1)           # LED ON
49         elif data.decode('utf-8') == '0':  # 0 received
50             GPIO.output(LED, 0)           # LED OFF
51
52 except KeyboardInterrupt:                 # Keyboard interrupt
53     client.close()
54     s.close()
55
56
```

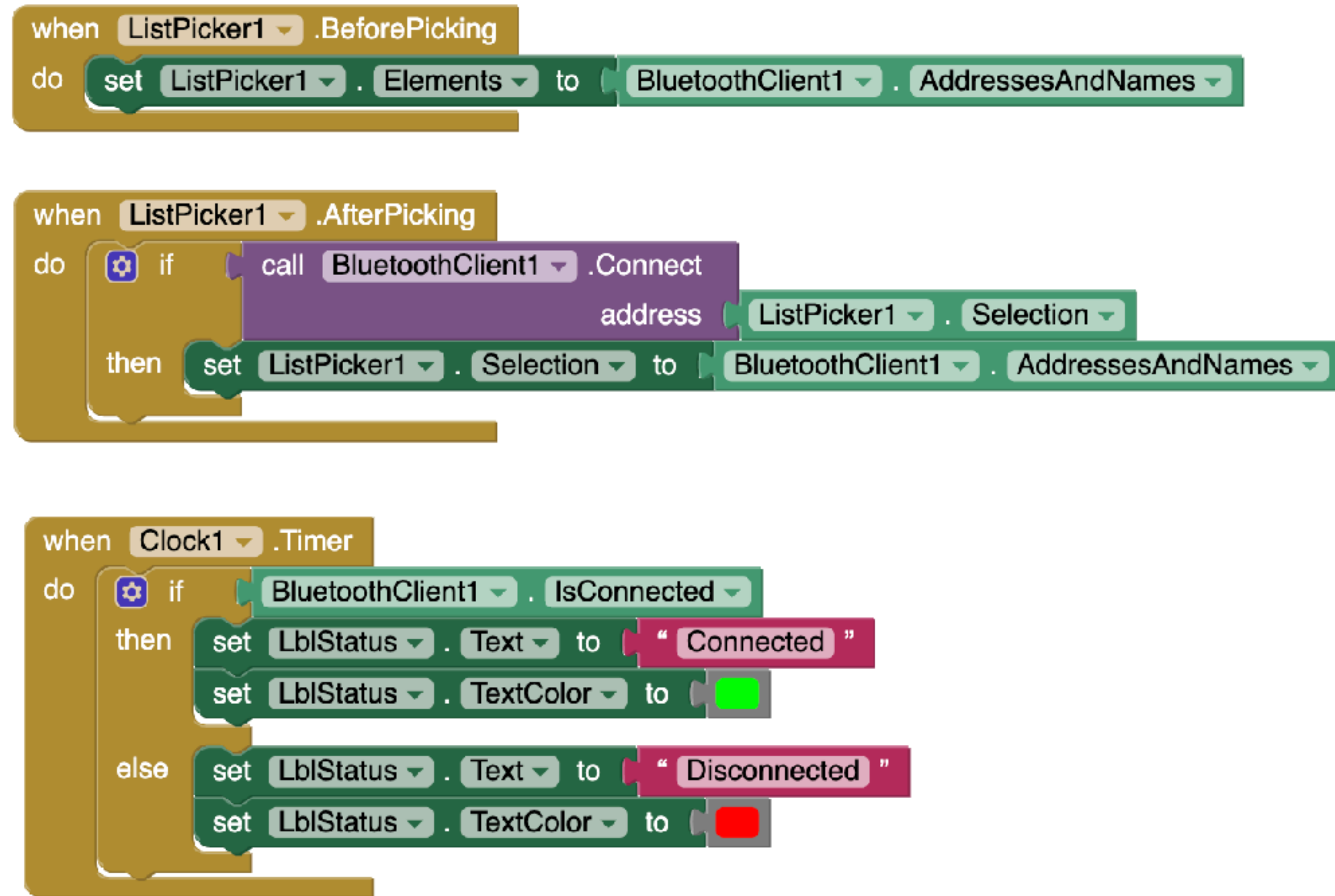

App Inventor - LED control through bluetooth

Application designer
build.



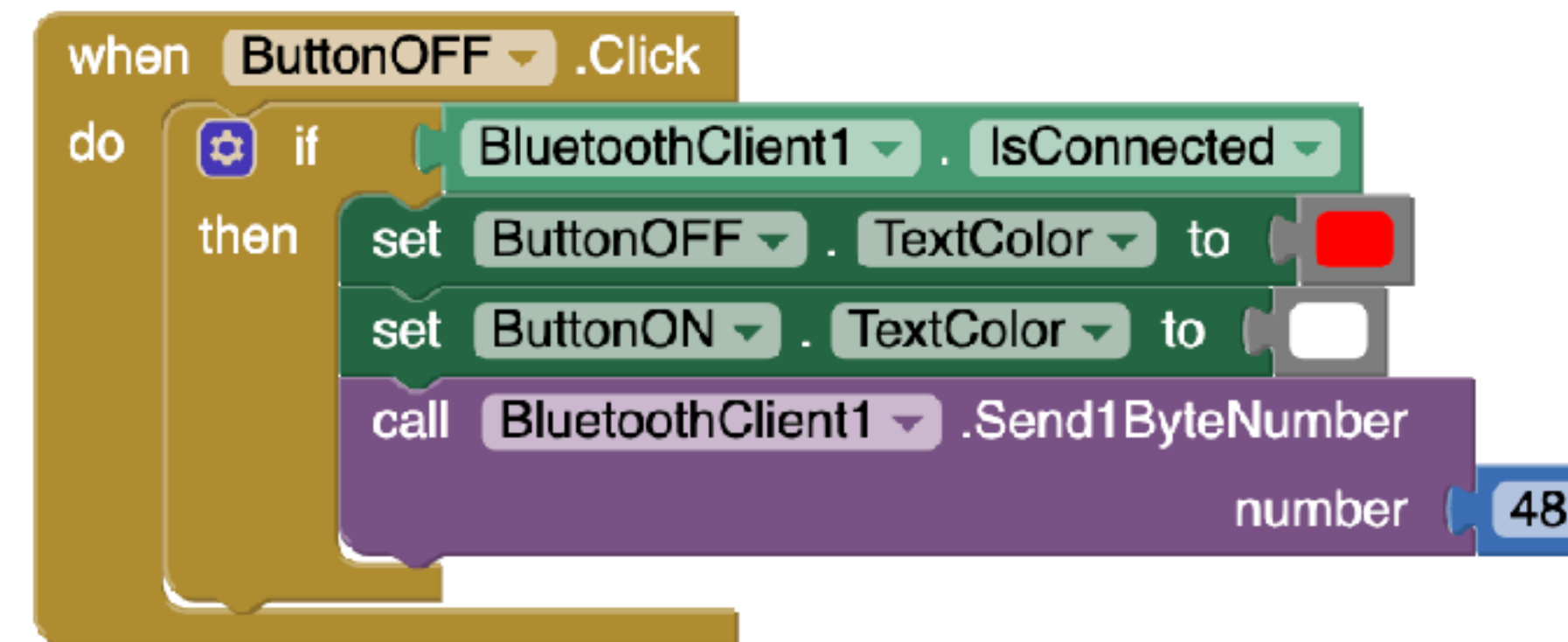
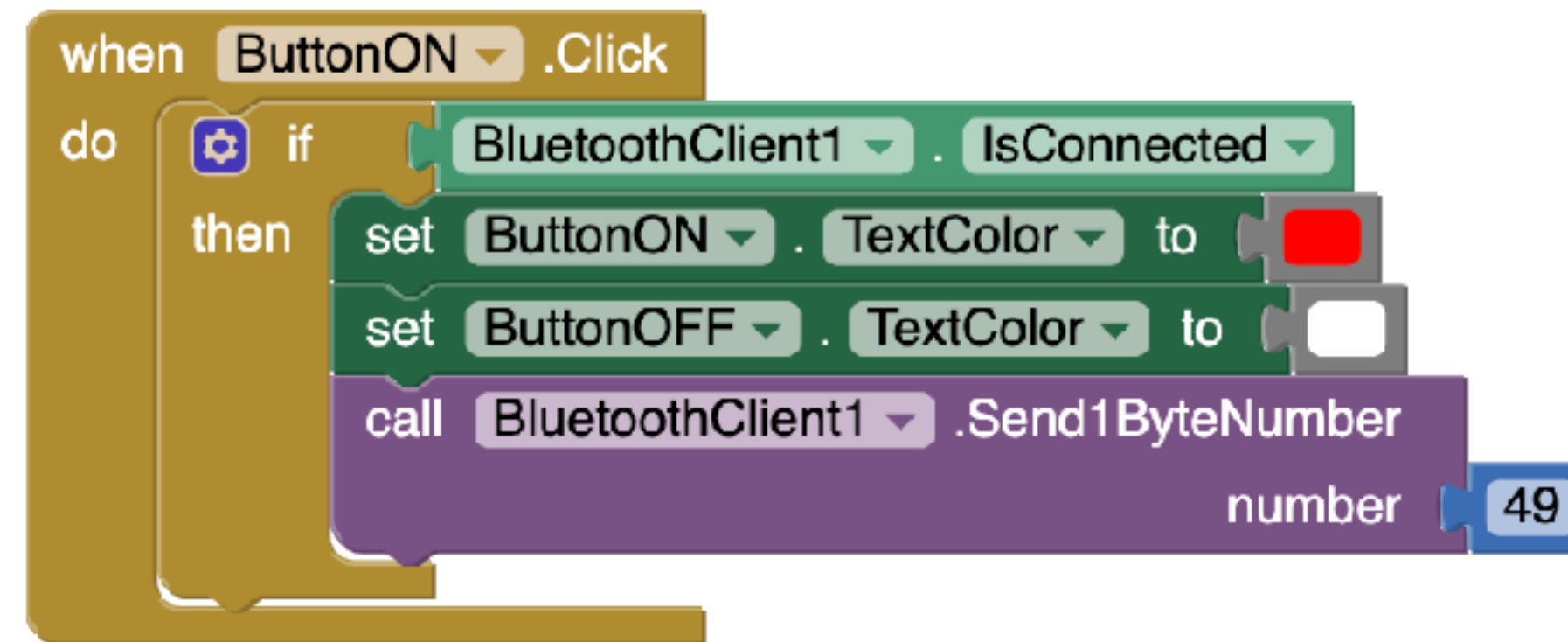
App Inventor - LED control through bluetooth

Application blocks
build.



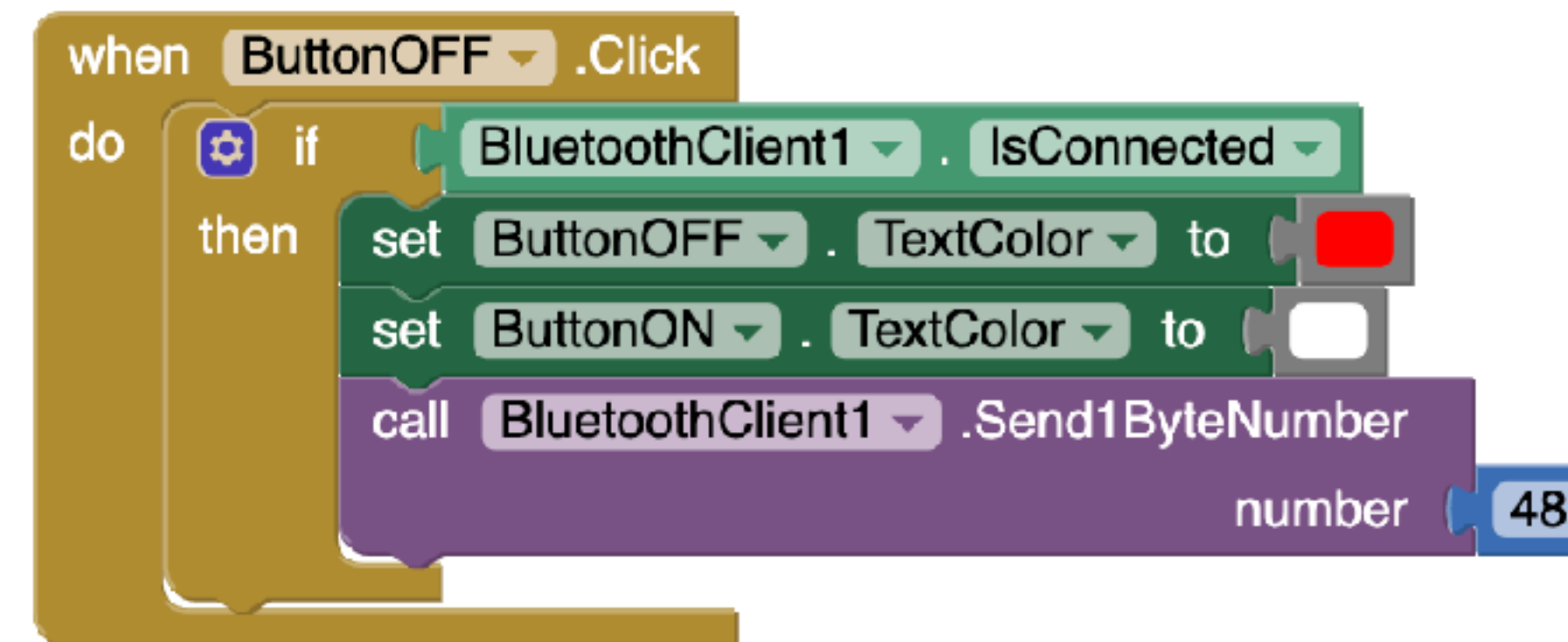
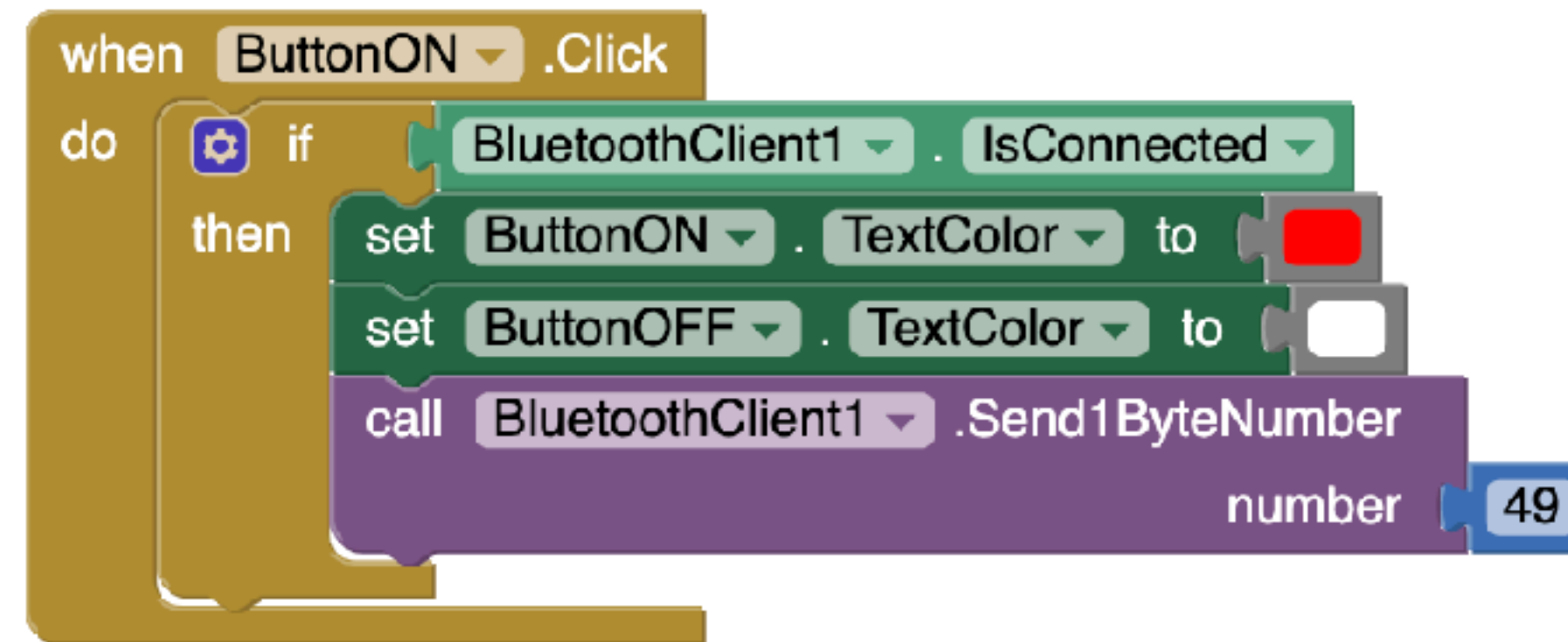
App Inventor - LED control through bluetooth

Application blocks
build.



App Inventor - LED control through bluetooth

Application blocks
build.



LED Control App - Bluetooth

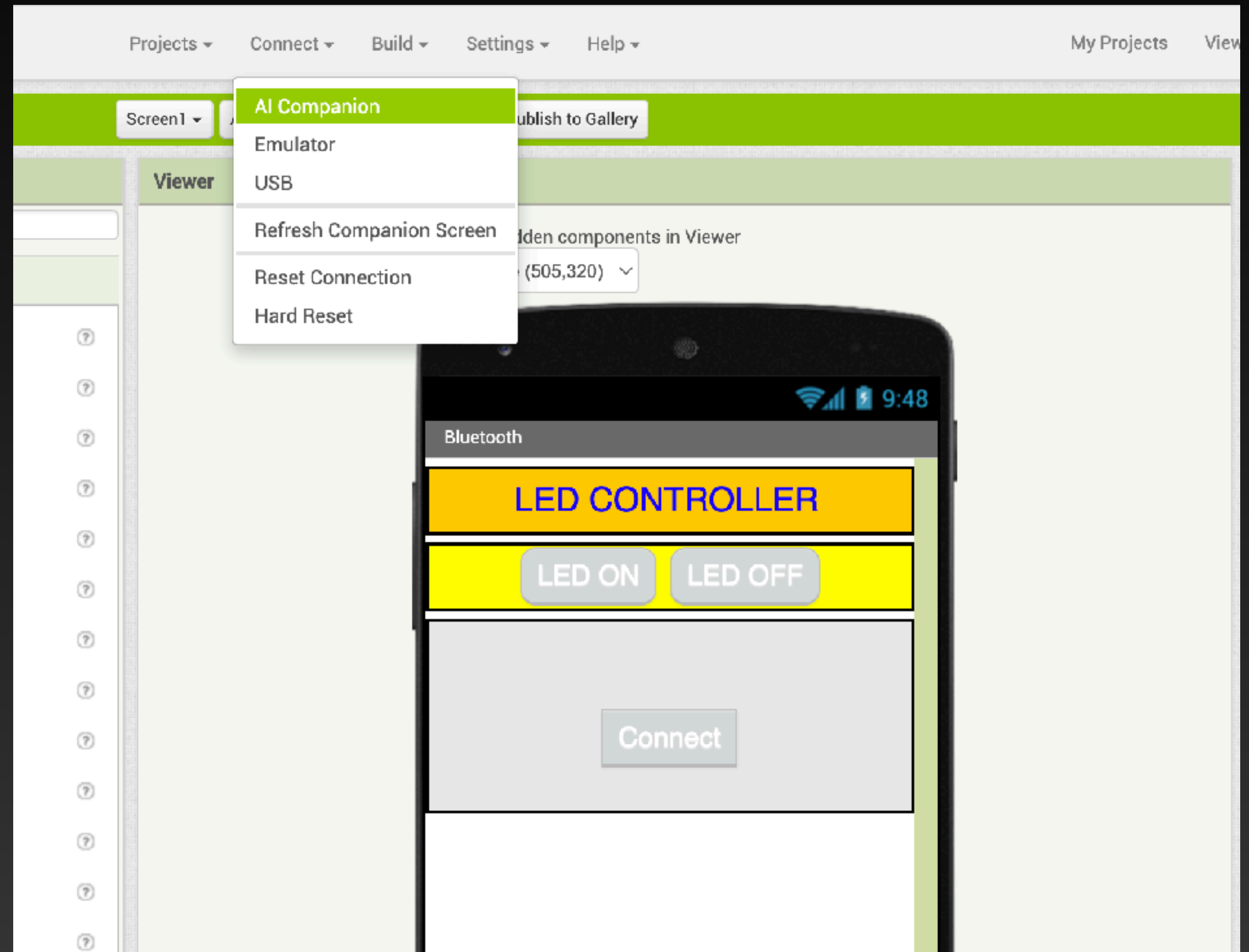
Test steps:

- Make sure that Bluetooth is enabled on your Android device.
- Make Bluetooth discoverable by clicking the Bluetooth icon on the top right-hand side of your raspberry Pi.
- You may have to accept to pair the two Bluetooth devices. If it asks for a password, enter 1234.
- Run your python program from GUI.

LED Control App - Bluetooth

Test steps - continued :

Generate QR code through AI companion, and scan from your android device.

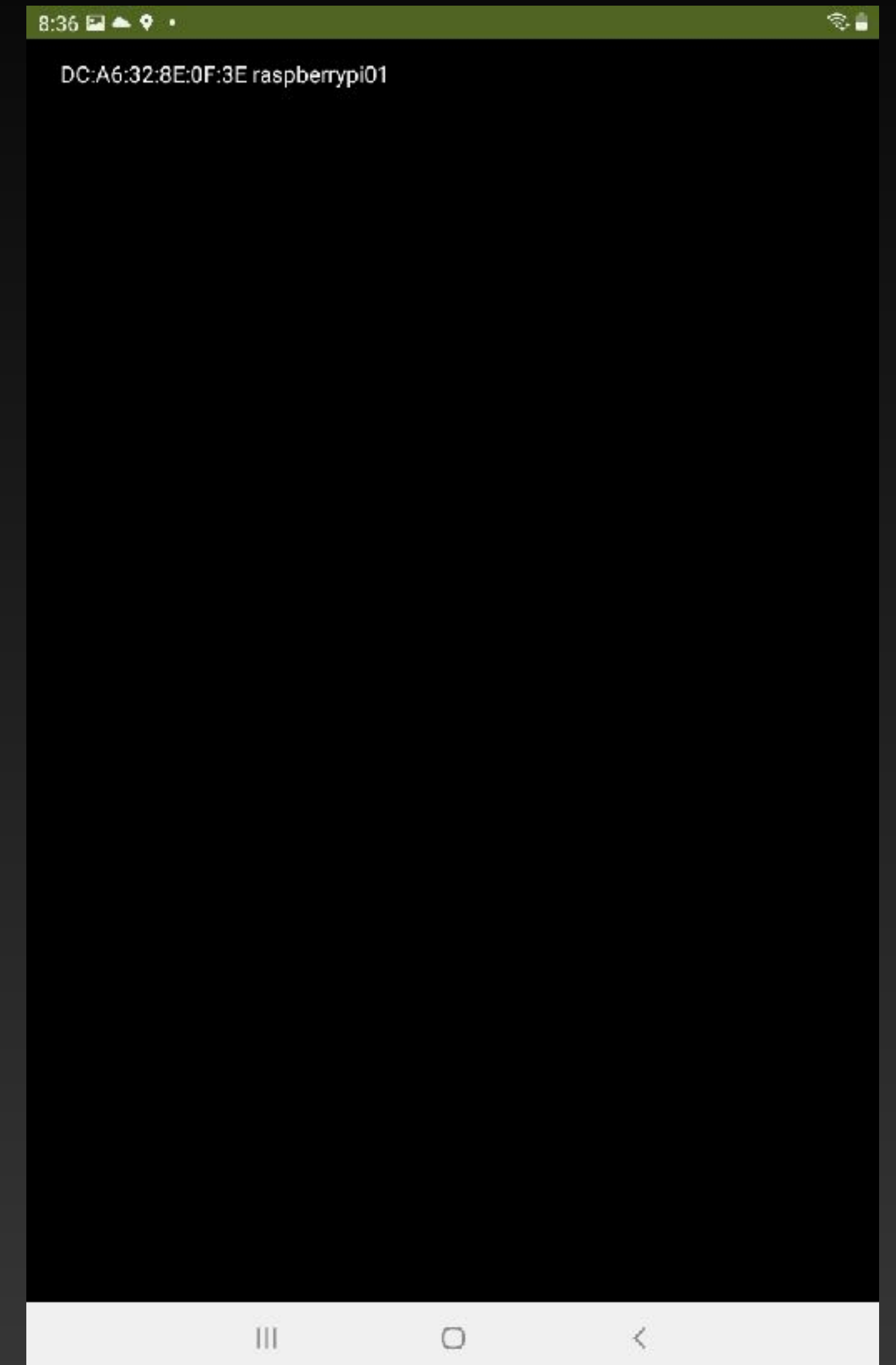
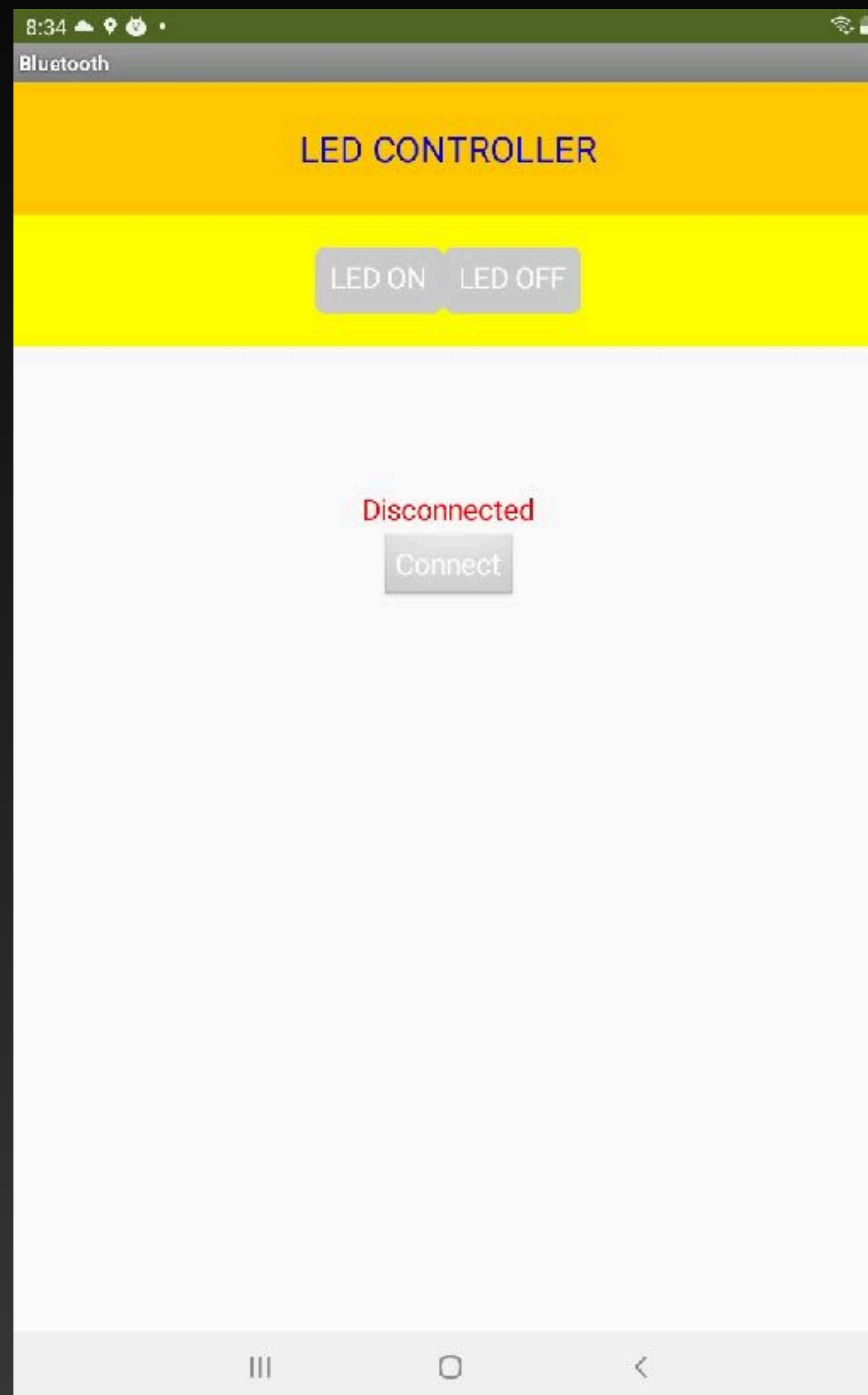


LED Control App - Bluetooth

Test steps - continued :

Start the mobile application on your Android device and click button Connect.

You should see the reachable bluetooth devices, click on raspberry Pi.



LED Control App - Bluetooth

Test steps - continued :

After click on raspberryPi,
your app will show
Connected!

Now you can test your
application.

